



UNIVERSITÀ DEGLI STUDI DI BERGAMO

Dipartimento di Ingegneria e Scienze Applicate

Corso di Laurea in Ingegneria Informatica

JarFin

*Sistema di Gestione Finanziaria Assistito da
Interazione Conversazionale*

Candidati:

Davide Bonsembiante (1086863)
Alessandro Biscaro (1087892)
Alessandro Rocco (1081179)

Corso:

Progettazione di Algoritmi
e Computabilità

Docente:

Prof.ssa Patrizia Scandurra

Anno Accademico:

2025 / 2026

Abstract

Il presente elaborato descrive la progettazione e lo sviluppo di *JarFin*, un sistema software per la gestione e l'analisi delle finanze personali dotato di interfaccia conversazionale. L'obiettivo del progetto è definire e realizzare un'architettura software modulare che integri servizi di gestione dei dati finanziari, componenti di analisi algoritmica e un'interfaccia basata su linguaggio naturale, garantendo coerenza, scalabilità e chiarezza nella separazione delle responsabilità tra i diversi moduli.

Il sistema è stato sviluppato seguendo una metodologia Agile, con particolare riferimento a SCRUM e all'approccio Agile Model Driven Development (AMDD), consentendo una progettazione incrementale basata su modelli e una validazione progressiva dei requisiti. L'architettura risultante è composta da servizi distinti, tra cui un servizio di accounting responsabile della gestione delle transazioni finanziarie e un servizio di analytics dedicato all'elaborazione, all'aggregazione e all'analisi dei dati, con particolare attenzione alla progettazione degli algoritmi e alla loro complessità computazionale.

Un elemento caratterizzante del sistema è l'integrazione di un'interfaccia conversazionale che consente all'utente di interrogare il sistema mediante linguaggio naturale. Questa componente introduce specifiche problematiche architetturali, tra cui la gestione dei flussi informativi tra servizi, la traduzione delle richieste in operazioni computazionali e la produzione di risposte coerenti e semanticamente corrette.

Il lavoro presenta inoltre una valutazione dell'architettura progettata e delle soluzioni implementate, evidenziandone i punti di forza e i limiti. JarFin rappresenta un caso di studio concreto per l'applicazione dei principi di progettazione algoritmica, modellazione del software e progettazione architetturale nella realizzazione di un sistema software complesso e modulare.

Indice

Abstract	1
1 Introduzione	8
1.1 Contesto e motivazione	8
1.2 Problema	9
1.3 Obiettivi del progetto	11
1.4 Soluzione proposta	12
1.5 Contributi del lavoro	13
2 Metodologia di sviluppo e tecnologie adottate	14
2.1 Metodologia di sviluppo	14
2.2 Organizzazione delle iterazioni	15
2.3 Architettura generale del sistema	16
2.4 Tecnologie adottate	16
2.4.1 Spring Boot	16
2.4.2 Database PostgreSQL e Containerizzazione	17
2.4.3 Modulo di Natural Language Understanding	17
2.5 Toolchain di progetto	18
2.5.1 Strumenti di modellazione	18
2.5.2 Versioning e Project Management	18
2.5.3 Sviluppo backend	19
2.5.4 Testing delle API REST	19
2.5.5 Unit Testing e Mocking	20
2.5.6 Analisi della copertura del codice	20

2.5.7	Analisi statica e architetturale	20
2.5.8	Ambienti di Sviluppo (IDE) e Build Automation	21
2.5.9	Considerazioni sulla toolchain	22
3	Analisi dei requisiti e architettura del sistema	23
3.1	Visione del prodotto	23
3.1.1	Ispirazione e contesto	23
3.1.2	Obiettivi del sistema	24
3.2	Analisi dei requisiti	24
3.2.1	Requisiti funzionali	24
3.2.2	Requisiti non funzionali	25
3.3	Strategia di modellazione architetturale	26
3.4	Vista degli scenari – Use Case Diagram	26
3.4.1	Attori	27
3.4.2	Descrizione dei Casi d’Uso	27
3.5	Vista logica – Class Diagram	28
3.5.1	Shared Kernel e Modello di Dominio	29
3.5.2	Web UI e NLU Service	29
3.5.3	Analytics Service	29
3.5.4	Accounting Service	30
3.5.5	Orchestrazione e API Gateway	30
3.6	Vista di sviluppo – Component Diagram	31
3.6.1	Client Layer	32
3.6.2	Application Layer	32
3.6.3	Data Layer	32
3.6.4	Dipendenze e Relazioni	33
3.7	Vista di processo – Sequence Diagram	33
3.7.1	Fase 1: Input e Comando Vocale	34
3.7.2	Fase 2: Caricamento Dashboard (Lettura e Analisi)	34
3.7.3	Fase 3: Feedback Finale	34
3.8	Vista fisica – Deployment Diagram	35

3.8.1	Architettura dei Nodi e Ambienti	35
3.8.2	Protocolli di Comunicazione	36
3.9	Conclusioni dell'Iterazione 0	36
4	Iterazione 1: Implementazione dell'Accounting Service	37
4.1	Introduzione e Obiettivi dell'Iterazione	37
4.2	Stack Tecnologico e Configurazione	38
4.3	Orchestrazione (Docker Compose)	38
4.3.1	Analisi delle Scelte Implementative	39
4.4	Architettura Software e Design Patterns	40
4.4.1	Modellazione del Dominio: La scelta di BigDecimal	40
4.4.2	Pattern DTO e Mapper	41
4.5	Logica di Business e Gestione Errori	41
4.5.1	Service Layer	41
4.5.2	Gestione Centralizzata delle Eccezioni	42
4.6	Esposizione API: TransactionController	43
4.7	Qualità del Codice e Strategia di Testing	43
4.7.1	Analisi Statica e Refactoring (SonarLint)	43
4.7.2	Unit Testing e Copertura (JUnit 5, Mockito & EclEmma)	44
4.7.3	Analisi Architetturale (CodeMR)	46
4.7.4	Integration Testing (Postman)	48
4.8	Conclusione Iterazione 1	49
5	Iterazione 2: Implementazione dell'Analytics Service	50
5.1	Obiettivi e Ruolo Architetturale	50
5.2	Stack Tecnologico e Configurazione	50
5.2.1	Configurazione del Client HTTP (RestTemplate)	51
5.2.2	Externalizzazione delle Dipendenze	51
5.3	Implementazione della Logica di Business	52
5.3.1	Modellazione dei Dati (Type Safety)	52
5.3.2	Algoritmo di Aggregazione (Design in Piccolo)	52
5.4	Analisi dell'Algoritmo di Analytics	53

5.4.1	Pseudocodice Formale	54
5.4.2	Analisi della Complessità Computazionale	55
5.4.3	Analisi della Complessità Spaziale	56
5.5	Algoritmi Predittivi e Sistema Esperto	57
5.5.1	Proiezione Temporale (Stima a fine mese)	57
5.5.2	Alert System (Savings Rate)	57
5.6	Qualità del Codice e Strategia di Testing	58
5.6.1	Analisi della Complessità e Refactoring	58
5.7	Testing Isolato con Mockito	59
5.7.1	Metriche di Copertura e Qualità	61
5.7.2	Validazione Funzionale del Servizio	63
5.8	Conclusione Iterazione 2	64
6	Iterazione 3: API Gateway, Web UI e NLU Engine	65
6.1	Obiettivi e Integrazione di Sistema	65
6.2	API Gateway: Centralizzazione del Routing	65
6.2.1	Configurazione Dichiarativa delle Rotte	66
6.3	Sviluppo dell'Interfaccia Utente (Web UI)	66
6.3.1	Dashboard Principale (index.html)	67
6.3.2	Gestione Storico (transactions.html)	67
6.4	Riconoscimento Vocale Ibrido	68
6.4.1	Strategia "Edge AI" (Client-Side Processing)	68
6.5	Natural Language Understanding Engine	69
6.5.1	Architettura della Pipeline di Parsing	70
6.5.2	Normalizzazione Morfologica e Semantica	70
6.5.3	Classificazione dell'Intento	71
6.5.4	Parsing Numerico Avanzato	72
6.5.5	Classificazione Semantica della Categoria	76
6.5.6	Determinazione del Tipo Finanziario	76
6.5.7	Generazione Intelligente della Descrizione	76
6.5.8	Costruzione dell'Oggetto ParsedTransaction	77

6.5.9	Pseudocodice dell'Algoritmo	77
6.5.10	Analisi della Complessità Computazionale	79
6.5.11	Robustezza Linguistica	79
6.6	Orchestrazione: CommandController	80
6.7	Strategia di Validazione e Testing	81
6.7.1	Analisi della Complessità (NLU Engine)	81
6.7.2	Testing di Integrazione e Resilienza	82
6.7.3	Validazione Architetturale: API Gateway	84
6.7.4	Validazione Funzionale: Flusso Utente (UX)	85
6.7.5	Metriche di Qualità Finali	88
6.8	Conclusione Iterazione 3	89
7	Analisi Architetturale del Sistema	90
7.1	Analisi delle Dipendenze	90
7.2	Salute del Codice (Code Health)	91
7.3	Metriche di Dettaglio e Assenza di Anomalie	92
7.4	Overview degli Attributi di Qualità	93
8	Conclusioni e Sviluppi Futuri	95
8.1	Sintesi del Lavoro Svolto	95
8.2	Raggiungimento degli Obiettivi	96
8.3	Analisi Critica delle Scelte Tecniche	96
8.3.1	Microservizi vs Monolite	96
8.3.2	NLU Deterministico vs Generativo	96
8.4	Limitazioni dell'Attuale Implementazione	97
8.4.1	Rigidità del Motore NLU	97
8.4.2	Modelli Predittivi Elementari	97
8.4.3	Gestione della Sicurezza	97
8.4.4	Persistenza dei Log e Monitoraggio	98
8.5	Sviluppi Futuri (Roadmap v2.0)	98
8.5.1	Sicurezza e Autenticazione (OAuth2)	98
8.5.2	Evoluzione del Motore NLU	98

8.5.3	Digitalizzazione Scontrini tramite OCR	98
8.5.4	Container Orchestration (Kubernetes)	99
8.6	Conclusione Personale	99

Capitolo 1

Introduzione

1.1 Contesto e motivazione

Negli ultimi anni, la progettazione di sistemi software complessi ha registrato una progressiva transizione da architetture monolitiche verso architetture distribuite basate su servizi indipendenti. Tale evoluzione è stata guidata dall'esigenza di realizzare sistemi più scalabili, manutenibili e capaci di adattarsi a requisiti in continua evoluzione.

In questo contesto, l'architettura a microservizi si è affermata come uno dei paradigmi più rilevanti (Figura 1.1). Essa propone la scomposizione di un'applicazione in un insieme di servizi autonomi, ciascuno responsabile di una specifica funzionalità e comunicante tramite interfacce ben definite. Questo approccio consente di ridurre l'accoppiamento tra componenti, facilitare l'evoluzione incrementale del sistema e migliorare la resilienza complessiva dell'architettura [1].

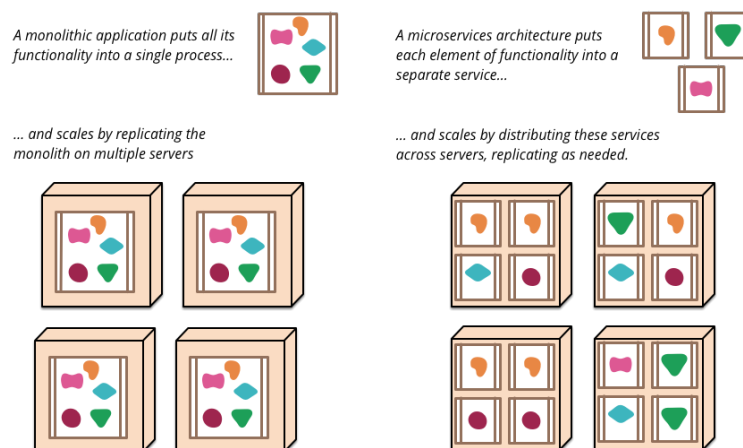


Figura 1.1: Architettura monolitica vs microservizi.

Parallelamente, la diffusione di tecnologie web standard ha favorito l'adozione di modelli architetturali basati su API REST (Representational State Transfer), che costituiscono uno stile architetturale ampiamente utilizzato per la comunicazione tra sistemi distribuiti. Le API REST si basano su interazioni stateless, risorse identificabili e interfacce uniformi, favorendo interoperabilità e semplicità di integrazione tra componenti eterogenei [2]. Un esempio di struttura distribuita che sfrutta questi principi è riportato in Figura 1.2.

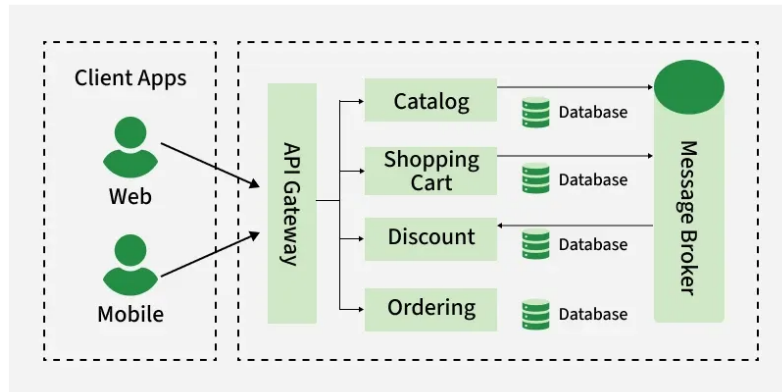


Figura 1.2: Struttura tipica di un'architettura a microservizi con API Gateway [3].

Un ulteriore sviluppo significativo riguarda l'integrazione di interfacce conversazionali, che consentono agli utenti di interagire con i sistemi software tramite linguaggio naturale. Questo paradigma migliora l'accessibilità e l'usabilità delle applicazioni, riducendo la complessità dell'interazione e rendendo i sistemi più intuitivi anche per utenti non esperti [4].

Nel dominio delle applicazioni per la gestione delle finanze personali, l'adozione combinata di architetture a microservizi, API REST e interfacce conversazionali apre nuove opportunità per la progettazione di sistemi modulari, scalabili e orientati all'utente, capaci di integrare funzionalità avanzate di analisi dei dati e di interazione.

1.2 Problema

I sistemi tradizionali per la gestione delle finanze personali presentano spesso limitazioni architetturali e funzionali. In particolare, molti di essi sono progettati come applicazioni monolitiche, caratterizzate da un forte accoppiamento tra componenti e da una limitata separazione delle responsabilità (Figura 1.3).

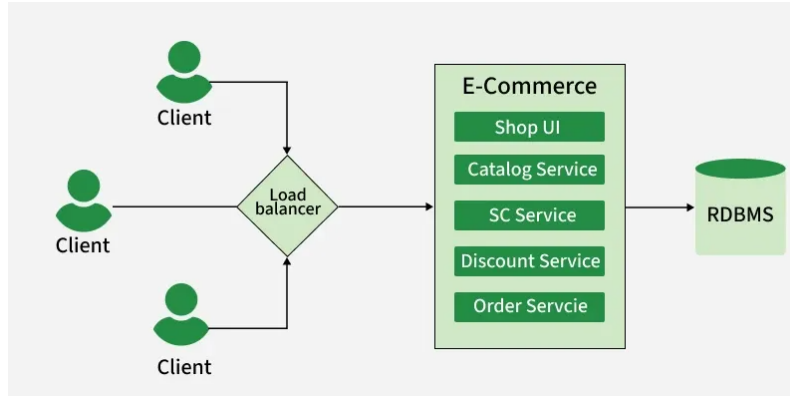


Figura 1.3: Esempio di struttura di un'applicazione e-commerce monolitica [3].

Questo approccio rende complessa l'evoluzione del sistema, ostacola la manutenzione del codice e limita l'introduzione di nuove funzionalità. Inoltre, l'interazione con tali sistemi avviene prevalentemente tramite interfacce grafiche tradizionali, che richiedono all'utente di navigare manualmente tra diverse schermate e inserire dati in modo strutturato. Questo approccio risulta spesso poco intuitivo e inefficiente, in particolare per utenti non esperti o in presenza di un elevato numero di operazioni da gestire. Dal punto di vista dell'esperienza utente, tali modalità di interazione introducono ulteriori criticità. L'inserimento manuale delle transazioni tramite form strutturati comporta un elevato carico cognitivo e operativo: la necessità di specificare categorie, importi e date per ogni singola operazione richiede tempo e attenzione, riducendo la probabilità di un utilizzo continuativo dell'applicazione nel lungo periodo (Figura 1.4).



Figura 1.4: Confronto tra il flusso di inserimento dati tradizionale e quello di JarFin.

In molti casi, l'utente è costretto ad adattarsi al modello dati del software, piuttosto che interagire secondo il proprio modello mentale. Manca infatti un approccio *frictionless* che consenta di registrare una spesa o interrogare i propri dati finanziari con la stessa naturalezza con cui tali informazioni vengono comunicate verbalmente. Dal punto di vista architetturale, la mancanza di modularità limita inoltre la scalabilità del sistema e ne riduce la robustezza, rendendo più complessa l'integrazione con nuovi servizi e l'evoluzione futura dell'applicazione.

Pertanto, emerge la necessità di progettare un sistema software che consenta di:

- gestire le finanze personali in modo strutturato e affidabile;
- garantire modularità e separazione delle responsabilità;
- supportare architetture distribuite scalabili;
- fornire un'interfaccia utente intuitiva, inclusa l'interazione tramite linguaggio naturale;
- consentire un'analisi efficiente dei dati finanziari.

1.3 Obiettivi del progetto

L'obiettivo del presente lavoro è la progettazione e lo sviluppo di *JarFin* (Java Artificial Financial Intelligence), un sistema software distribuito per la gestione e l'analisi delle finanze personali, basato su un'architettura a microservizi.

Gli obiettivi specifici del progetto includono:

- la progettazione di un'architettura software modulare basata su microservizi;
- la realizzazione di servizi indipendenti per la gestione delle transazioni e l'analisi dei dati;
- l'implementazione di un sistema di comunicazione basato su API REST;
- l'integrazione di un modulo di Natural Language Understanding per l'elaborazione di comandi in linguaggio naturale;
- la realizzazione di un'interfaccia web per l'interazione con il sistema;
- l'applicazione di principi di progettazione algoritmica e modellazione software.

1.4 Soluzione proposta

Per soddisfare gli obiettivi definiti, il sistema è stato principalmente suddiviso nei seguenti componenti (Figura 1.5):

- **Web Application**, che rappresenta l'interfaccia utente e consente l'interazione con il sistema tramite browser;
- **API Gateway**, che funge da punto di accesso centralizzato e gestisce l'instradamento delle richieste verso i microservizi appropriati;
- **Accounting Service**, responsabile della logica di business relativa alla gestione delle transazioni finanziarie e della validazione dei dati;
- **Analytics Service**, che elabora i dati finanziari storici per generare report e statistiche aggregate;
- **NLU Service**, che consente l'elaborazione di comandi in linguaggio naturale e la loro traduzione in operazioni strutturate;
- **Database PostgreSQL**, istanza relazionale centralizzata che garantisce la persistenza, l'integrità referenziale e la condivisione dei dati tra i servizi funzionali.

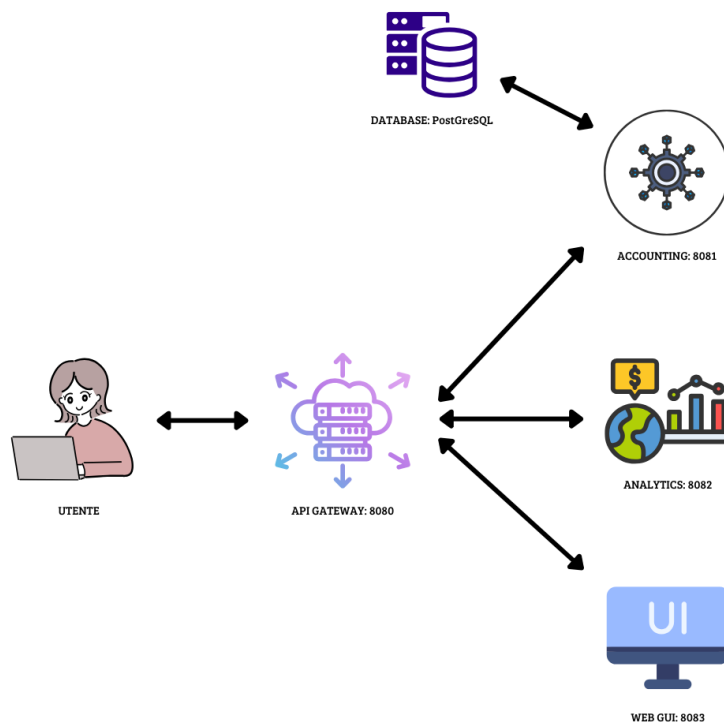


Figura 1.5: Architettura specifica proposta per il progetto JarFin.

La comunicazione tra i componenti avviene tramite API REST e scambio di dati in formato JSON, secondo i principi architetturali definiti nello stile REST [2]. L'utilizzo del framework Spring Boot consente lo sviluppo di microservizi indipendenti e facilmente configurabili, migliorando la modularità e la manutenibilità complessiva del sistema [5].

L'architettura complessiva del sistema è illustrata nei diagrammi presenti nei capitoli successivi, che evidenziano le relazioni tra i diversi microservizi e il ruolo centrale dell'API Gateway.

1.5 Contributi del lavoro

I principali contributi del presente lavoro includono:

- la progettazione e validazione di un'architettura software distribuita basata su microservizi per la gestione delle finanze personali;
- l'integrazione di un modulo di Natural Language Understanding in un sistema finanziario reale, consentendo l'interazione tramite linguaggio naturale;
- la realizzazione di un flusso completo di acquisizione, elaborazione e analisi dei dati finanziari tramite servizi indipendenti;
- l'adozione di un'architettura basata su API REST e API Gateway per migliorare la scalabilità e ridurre l'accoppiamento tra componenti;
- l'applicazione di principi di progettazione algoritmica e architetturale in un contesto concreto e funzionante;
- la validazione dell'approccio proposto attraverso implementazione e testing.

Capitolo 2

Metodologia di sviluppo e tecnologie adottate

2.1 Metodologia di sviluppo

Lo sviluppo del sistema *JarFin* è stato condotto adottando una metodologia Agile, con l'obiettivo di favorire un processo di progettazione incrementale, flessibile e orientato al cambiamento. In particolare, il progetto ha seguito i principi di **SCRUM**, integrati con l'approccio **Agile Model Driven Development (AMDD)**, al fine di bilanciare la necessità di una progettazione architetturale solida con i vantaggi dell'iterazione continua e del feedback costante.

SCRUM è stato utilizzato come framework di riferimento per l'organizzazione del lavoro, la pianificazione delle attività e la gestione delle iterazioni di sviluppo. Il ciclo di vita del progetto è stato articolato in sprint di durata indicativa pari a due settimane, ciascuno caratterizzato da obiettivi ben definiti e dalla produzione di un incremento funzionale del sistema. Questa impostazione ha consentito di monitorare costantemente l'avanzamento del progetto e di adattare le scelte progettuali in base alle esigenze emerse durante lo sviluppo [6].

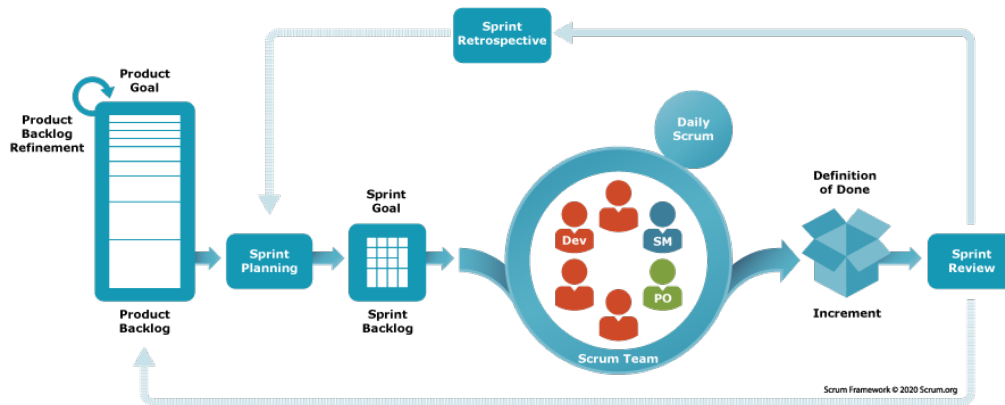


Figura 2.1: Il framework SCRUM e il ciclo degli Sprint.

L'approccio AMDD ha guidato le attività di modellazione e progettazione. Invece di definire in modo esaustivo l'intera architettura nelle fasi iniziali, la modellazione è stata condotta in maniera incrementale, producendo modelli essenziali e sufficienti a supportare le decisioni architetturali di ciascuna iterazione. In particolare, sono stati realizzati diagrammi UML per i casi d'uso più rilevanti, successivamente raffinati nel corso dello sviluppo. Questo approccio ha permesso di evitare un'eccessiva rigidità progettuale, mantenendo una visione globale del sistema.

2.2 Organizzazione delle iterazioni

Il ciclo di sviluppo di JarFin è stato articolato in una serie di iterazioni successive, ciascuna focalizzata su specifici obiettivi funzionali e architetturali:

- **Iterazione 0 – Envisioning e Analisi:** definizione del dominio applicativo, raccolta e analisi dei requisiti funzionali e non funzionali, identificazione dei principali casi d'uso e progettazione dell'architettura di alto livello;
- **Iterazione 1 – Core Services:** sviluppo del servizio di accounting e implementazione della persistenza dei dati.
- **Iterazione 2 – Analytics e Algoritmi:** sviluppo del servizio di analytics e implementazione degli algoritmi per l'elaborazione e l'analisi dei dati finanziari;
- **Iterazione 3 – Integrazione e Interazione:** integrazione del modulo di Natural Language Understanding, dell'API Gateway e dell'interfaccia web.

Questa suddivisione ha consentito di affrontare progressivamente la complessità del sistema, validando le scelte progettuali a ogni iterazione e riducendo il rischio di errori architetturali nelle fasi avanzate dello sviluppo.

2.3 Architettura generale del sistema

L'architettura di JarFin è basata su un paradigma a microservizi, in cui il sistema è scomposto in un insieme di componenti indipendenti, ciascuno responsabile di una specifica area funzionale. Tale scelta architetturale è motivata dall'esigenza di garantire modularità, scalabilità e manutenibilità, oltre a facilitare l'evoluzione futura del sistema.

Ogni microservizio è progettato come un'unità autonoma, dotata di una propria logica applicativa e, ove necessario, di un proprio livello di persistenza. La comunicazione tra i servizi avviene tramite API REST e scambio di messaggi in formato JSON, secondo i principi dello stile architetturale REST [2].

L'adozione di un'architettura distribuita consente inoltre di isolare le responsabilità dei singoli componenti, riducendo l'impatto delle modifiche e migliorando la resilienza complessiva del sistema in caso di malfunzionamenti localizzati.

2.4 Tecnologie adottate

La scelta dello stack tecnologico è stata guidata dalla necessità di utilizzare strumenti robusti, ampiamente supportati dalla community e adatti allo sviluppo di sistemi distribuiti.

2.4.1 Spring Boot

Per l'implementazione dei microservizi è stato utilizzato il framework **Spring Boot**, che fornisce un ambiente di sviluppo semplificato per la realizzazione di applicazioni Java basate su architettura a servizi. Spring Boot consente di ridurre significativamente la configurazione manuale, favorendo lo sviluppo rapido di applicazioni stand-alone e facilmente deployabili [5].

Il framework supporta nativamente la creazione di API REST, l'integrazione con sistemi di persistenza e la gestione della configurazione dei servizi, risultando particolarmente adatto allo sviluppo di architetture a microservizi.

2.4.2 Database PostgreSQL e Containerizzazione

La persistenza dei dati è affidata a un database relazionale **PostgreSQL**, scelto per la sua affidabilità, conformità agli standard SQL e supporto a funzionalità avanzate di gestione dei dati. PostgreSQL garantisce l'integrità delle transazioni finanziarie (proprietà ACID) e supporta operazioni di interrogazione efficienti, fondamentali per le funzionalità di analisi offerte dal sistema.

Per semplificare la gestione dell'ambiente di sviluppo e garantire l'isolamento dei dati, l'istanza del database è stata containerizzata tramite **Docker**. Questa scelta architetturale consente di avviare il servizio di persistenza in modo rapido e indipendente dalla configurazione del sistema operativo ospite, evitando installazioni locali invasive e garantendo un ambiente sempre pulito e riproducibile (Figura 2.2).

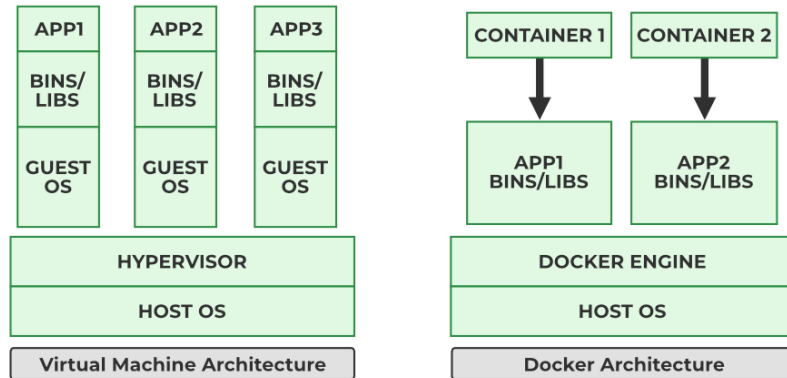


Figura 2.2: Confronto architetturale tra Macchine Virtuali e Container Docker [7].

2.4.3 Modulo di Natural Language Understanding

Per l'elaborazione del linguaggio naturale è stato sviluppato un modulo di Natural Language Understanding implementato in Java come microservizio dedicato. Il sistema utilizza tecniche di analisi lessicale e *pattern matching* per identificare intenti e parametri rilevanti a partire dalle frasi dell'utente, consentendo l'esecuzione di operazioni finanziarie tramite comandi in linguaggio naturale.

La scelta di una soluzione interna, anziché basata su servizi cloud esterni, consente di ridurre la latenza, mantenere il controllo sui dati sensibili dell'utente e garantire una maggiore integrazione con l'architettura complessiva del sistema.

2.5 Toolchain di progetto

Lo sviluppo del sistema *JarFin* è stato supportato da una toolchain articolata, composta da strumenti dedicati alla modellazione, allo sviluppo, al versioning, al testing e all'analisi della qualità del codice. La scelta degli strumenti è stata guidata dall'obiettivo di garantire un processo di sviluppo strutturato, tracciabile e conforme alle buone pratiche dell'ingegneria del software.

2.5.1 Strumenti di modellazione

Per la modellazione del sistema è stato utilizzato **UMLet**, uno strumento leggero per la creazione di diagrammi UML. UMLet è stato impiegato per la realizzazione dei principali artefatti di modellazione previsti dall'approccio AMDD, in particolare:

- diagrammi dei casi d'uso, per rappresentare le funzionalità offerte dal sistema e le interazioni con gli attori;
- diagrammi delle classi, per descrivere la struttura statica dei principali componenti software;
- diagrammi dei componenti, per modellare l'architettura a microservizi;
- diagrammi di sequenza, per analizzare il comportamento dinamico del sistema nei casi d'uso critici.
- diagrammi di deployment, per descrivere la distribuzione fisica dei nodi e l'architettura containerizzata del database.

L'utilizzo di UMLet ha consentito di produrre modelli essenziali e facilmente modificabili, in linea con i principi della modellazione incrementale promossi dall'Agile Model Driven Development.

2.5.2 Versioning e Project Management

Il controllo di versione del codice sorgente e la gestione del progetto sono stati affidati a **GitHub**. Git è stato utilizzato come sistema di versioning distribuito, permettendo di tracciare l'evoluzione del codice, gestire rami di sviluppo separati e mantenere uno storico completo delle modifiche.

GitHub è stato inoltre impiegato come strumento di supporto al project management, sfruttando:

- repository centralizzato per il codice sorgente;
- Kanban board per la pianificazione delle attività e il tracciamento delle iterazioni;
- Pull Request per la revisione del codice e l'integrazione controllata delle funzionalità.

Questa impostazione ha favorito un processo di sviluppo ordinato e trasparente, coerente con la metodologia SCRUM adottata.

2.5.3 Sviluppo backend

Il backend del sistema è stato sviluppato utilizzando il linguaggio **Java** e il framework **Spring Boot**. Spring Boot è stato scelto per la sua capacità di semplificare lo sviluppo di applicazioni basate su architettura a microservizi, riducendo la configurazione manuale e favorendo la realizzazione di servizi REST stand-alone.

Ogni microservizio di JarFin è implementato come applicazione Spring Boot indipendente, responsabile di una specifica area funzionale del sistema, in linea con i principi di separazione delle responsabilità e modularità.

2.5.4 Testing delle API REST

Per la verifica manuale e il testing degli endpoint REST è stato utilizzato **Postman**. Lo strumento ha permesso di:

- testare le operazioni CRUD esposte dai microservizi;
- validare il corretto formato delle richieste e delle risposte JSON;
- simulare scenari di utilizzo reali durante le fasi di sviluppo e integrazione.

L'uso di Postman ha facilitato l'individuazione precoce di errori di comunicazione tra i servizi e ha supportato la fase di debugging dell'API Gateway.

2.5.5 Unit Testing e Mocking

Il testing automatico della logica applicativa è stato realizzato tramite **JUnit 5**, framework di riferimento per il testing in ambiente Java, affiancato da **Mockito**, un framework di mocking essenziale per isolare le unità di codice. Mentre JUnit è stato utilizzato per definire il ciclo di vita dei test e le asserzioni, Mockito ha permesso di simulare il comportamento delle dipendenze esterne (mock), garantendo che i test unitari si focalizzassero esclusivamente sulla logica interna dei servizi, in particolare nei moduli di accounting, analytics e NLU.

L'adozione combinata di questi strumenti ha contribuito a migliorare l'affidabilità del sistema e a ridurre il rischio di regressioni durante l'evoluzione del codice.

2.5.6 Analisi della copertura del codice

Per l'analisi della copertura dei test è stato utilizzato il plugin **EclEmma** (basato sulla libreria **JaCoCo**), integrato direttamente nell'ambiente di sviluppo. Lo strumento consente di misurare dinamicamente la percentuale di codice eseguito durante l'esecuzione dei test (code coverage), fornendo indicatori visivi e quantitativi utili per valutare l'efficacia della suite di test.

Le metriche di copertura hanno supportato l'identificazione delle porzioni di codice critiche non ancora verificate, guidando il miglioramento progressivo dei test unitari.

2.5.7 Analisi statica e architetturale

La qualità del software è stata monitorata su due livelli distinti mediante l'uso combinato di strumenti di analisi statica:

Qualità del Codice (SonarLint)

La qualità strutturale e la conformità agli standard sono state analizzate mediante **SonarLint**. Lo strumento è stato utilizzato per:

- rilevare metriche di complessità cognitiva e ciclomatica puntuale;
- individuare *code smells*, bug potenziali e vulnerabilità di sicurezza;
- supportare la verifica del rispetto dei principi di Clean Code.

Qualità Architettuale (CodeMR)

Per un’analisi di più alto livello, è stato introdotto **CodeMR**, uno strumento di analisi statica avanzata focalizzato sulla struttura del software. A differenza dei linter classici, CodeMR ha permesso di visualizzare e mappare le dipendenze tra i package, calcolando metriche architetture fondamentali quali:

- **Accoppiamento (Coupling)**: per valutare l’indipendenza dei moduli;
- **Complessità Ciclomantica**: aggregata a livello di classe e package;
- **Coesione**: per verificare la responsabilità singola delle classi.

L’utilizzo di CodeMR è stato determinante per verificare l’assenza di dipendenze cicliche tra i microservizi e per identificare preventivamente potenziali “God Classes” (classi monolitiche con troppe responsabilità), facilitando il refactoring verso un’architettura più modulare.

2.5.8 Ambienti di Sviluppo (IDE) e Build Automation

Per la gestione del ciclo di vita del software sono stati utilizzati due ambienti di sviluppo integrati (IDE) complementari, scelti in base alle specifiche esigenze delle diverse fasi di lavoro:

- **Visual Studio Code**: utilizzato come editor versatile per le attività di supporto e modellazione. Nello specifico, grazie all’integrazione dell’estensione *UMLet*, è stato impiegato per la realizzazione dei diagrammi UML. È stato inoltre lo strumento di riferimento per la redazione della documentazione tecnica (LaTeX/Markdown) e per la gestione delle operazioni di versioning tramite l’integrazione diretta con GitHub.
- **Eclipse**: adottato come IDE principale per lo sviluppo sia del backend che del frontend. È stato utilizzato per la scrittura del codice sorgente Java, la gestione delle risorse web, il debugging avanzato dei microservizi e l’esecuzione dei test unitari.

La build automation e la gestione delle dipendenze del progetto Java sono state affidate ad **Apache Maven**, garantendo la riproducibilità del processo di compilazione e una gestione centralizzata delle librerie su diverse macchine di sviluppo.

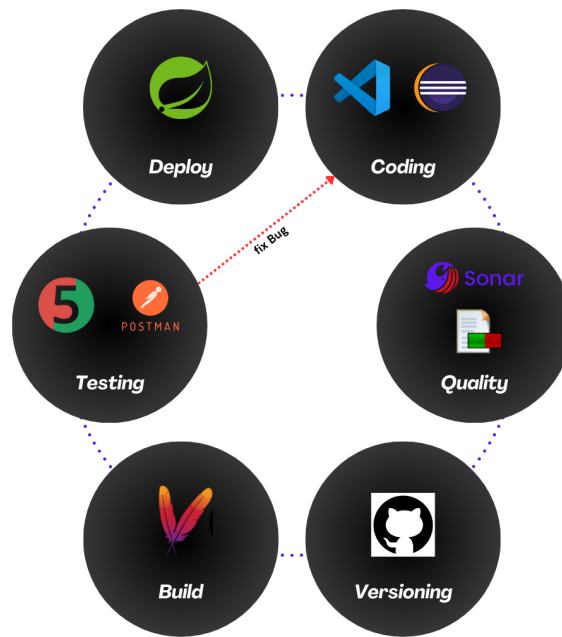


Figura 2.3: Il flusso di lavoro e la toolchain integrata del progetto JarFin.

2.5.9 Considerazioni sulla toolchain

L'integrazione degli strumenti descritti ha permesso di coprire l'intero ciclo di vita del software, dalla modellazione iniziale alla verifica della qualità del codice. La toolchain adottata rappresenta una soluzione completa e coerente con gli obiettivi del progetto, fornendo un supporto efficace sia allo sviluppo incrementale sia alla validazione tecnica del sistema (Figura 2.3).

Capitolo 3

Analisi dei requisiti e architettura del sistema

3.1 Visione del prodotto

3.1.1 Ispirazione e contesto

L'idea alla base del sistema *JarFin* trae ispirazione dal paradigma degli assistenti intelligenti proattivi, resi popolari in ambito cinematografico da sistemi come J.A.R.V.I.S. (*Just A Rather Very Intelligent System*). In tale contesto ideale, l'assistente è in grado di comprendere richieste espresse in linguaggio naturale e di eseguire operazioni complesse in modo autonomo, riducendo drasticamente il carico cognitivo dell'utente.

Nel mondo reale, tuttavia, la gestione delle finanze personali rappresenta un'attività spesso inefficiente e frammentata. Gli utenti sono costretti a utilizzare molteplici applicazioni bancarie, inserire manualmente dati in fogli di calcolo e navigare interfacce complesse (WIMP) per ottenere informazioni basilari sul proprio stato finanziario. Questa modalità operativa introduce un'elevata "frizione", aumentando il rischio di errori di trascrizione e riducendo la frequenza con cui gli utenti monitorano le proprie finanze.

3.1.2 Obiettivi del sistema

JarFin si propone di colmare questo divario introducendo un assistente finanziario intelligente in grado di:

- comprendere comandi espressi in linguaggio naturale (NLU);
- trasformare input non strutturati (es. "Pizza 20 euro") in operazioni finanziarie formalmente definite;
- centralizzare la gestione delle transazioni finanziarie in un unico hub;
- fornire funzionalità avanzate di analisi e reporting automatico;
- garantire un'architettura scalabile, modulare ed estendibile tramite microservizi.

3.2 Analisi dei requisiti

In questa fase di *Envisioning* (Iterazione 0), sono stati formalizzati i requisiti necessari per costruire il Core, l'Analytics e il modulo NLU.

3.2.1 Requisiti funzionali

I requisiti funzionali definiscono le capacità operative che il sistema deve esporre.

ID	Requisito	Descrizione	Priorità
RF-01	Gestione transazioni (CRUD)	Il sistema deve consentire la creazione, lettura, aggiornamento e cancellazione delle transazioni tramite API REST esposte dal servizio Accounting.	Alta
RF-02	Parsing del linguaggio naturale	Il servizio NLU deve interpretare input testuali ed estrarre le entità rilevanti (importo, categoria, tipo) utilizzando la classe <code>NaturalLanguageService</code> e il metodo <code>parse()</code> .	Alta

ID	Requisito	Descrizione	Priorità
RF-03	Aggregazione e reporting	Il servizio Analytics deve aggregare i dati finanziari e generare statistiche temporali tramite la classe <code>AnalyticsService</code> .	Media
RF-04	Orchestrazione API Gateway	Il sistema deve fornire un punto di accesso unico tramite <code>JarfinApiGatewayApplication</code> , responsabile del routing verso i microservizi.	Alta
RF-05	Output strutturato	Il sistema deve restituire risposte in formato JSON standardizzato per garantire interoperabilità con i client.	Media

3.2.2 Requisiti non funzionali

I requisiti non funzionali definiscono gli attributi di qualità del sistema.

ID	Requisito	Descrizione	Priorità
RNF-01	Modularità	Il sistema deve essere implementato con architettura a microservizi (Spring Boot), garantendo disaccoppiamento.	Alta
RNF-02	Manutenibilità	Il codice deve rispettare il pattern <i>Controller-Service-Repository</i> e i principi Clean Code.	Alta
RNF-03	Scalabilità	L'architettura deve supportare l'aggiunta di nuovi moduli (es. Speech-to-Text) senza rifattorizzazione.	Media
RNF-04	Usabilità	Il sistema deve garantire tempi di risposta adeguati per operazioni sincrone standard e supportare query in linguaggio naturale.	Alta
RNF-05	Integrità dei dati	La persistenza deve essere affidata a un DBMS relazionale (PostgreSQL) conforme alle proprietà ACID.	Alta

3.3 Strategia di modellazione architetturale

Per gestire la complessità di un sistema distribuito, è stato adottato il modello **UML 4+1 View Model**. Questo approccio consente di descrivere l'architettura attraverso cinque viste concorrenti:

- **Vista Scenari:** descrive le interazioni utente-sistema (Casi d'Uso).
- **Vista Logica:** descrive la struttura statica e il dominio (Classi).
- **Vista di Sviluppo:** descrive l'organizzazione dei moduli software (Componenti).
- **Vista di Processo:** descrive il comportamento dinamico e i flussi (Sequenza).
- **Vista Fisica:** descrive il deployment sull'infrastruttura (Deployment).

3.4 Vista degli scenari – Use Case Diagram

La vista degli scenari (Figura 3.1) guida l'intero processo di sviluppo, definendo cosa il sistema deve fare per gli attori coinvolti.

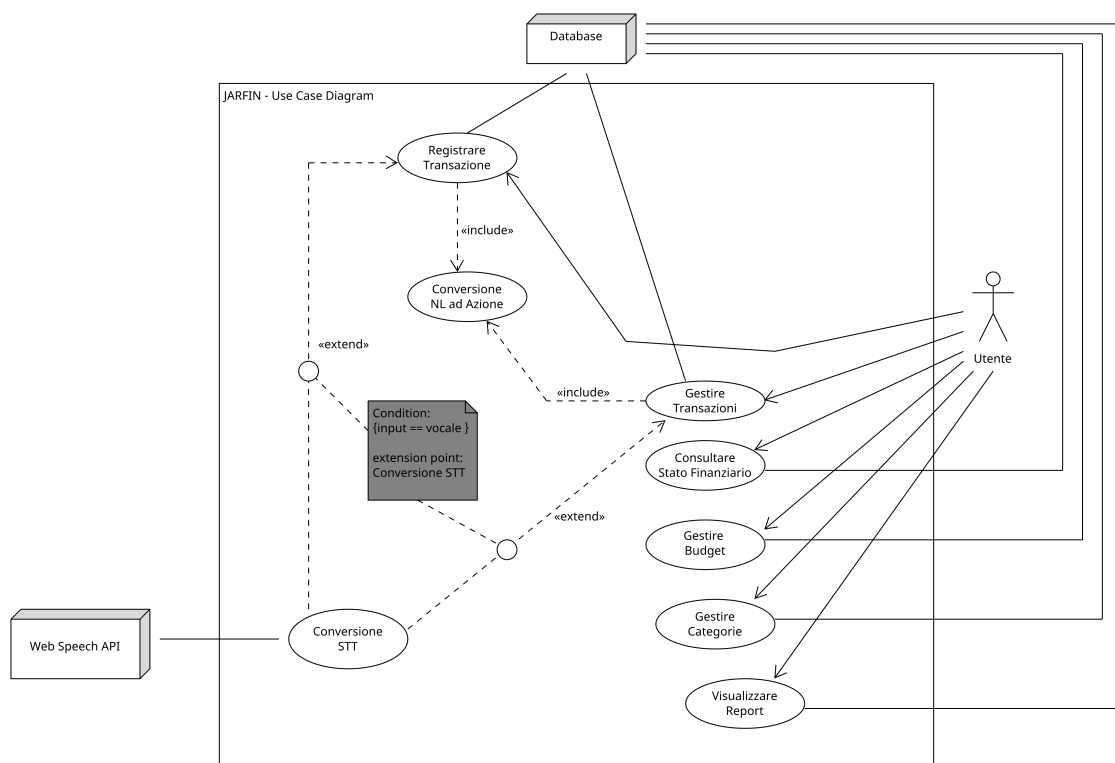


Figura 3.1: Diagramma dei casi d'uso (Vista degli Scenari).

3.4.1 Attori

Il diagramma identifica due tipologie di attori:

- **Utente:** attore primario che interagisce con l'interfaccia web per la gestione delle proprie finanze.
- **Sistemi esterni:** attori secondari necessari al funzionamento del backend:
 - **Cloud Speech-to-Text:** servizio invocato per la trascrizione dei comandi vocali;
 - **Database:** sistema responsabile della persistenza fisica di tutte le entità (utenti, transazioni, categorie).

3.4.2 Descrizione dei Casi d'Uso

Le funzionalità sono state raggruppate in tre macro-aree logiche per facilitarne la lettura e l'implementazione.

Gestione Transazioni

Rappresenta il nucleo operativo dell'applicazione (*Core Domain*).

- **Registrare Transazione:** permette l'inserimento di una nuova spesa o entrata.
 - *Inclusione («include»)*: include obbligatoriamente il caso d'uso "**Conversione NL ad Azione**", poiché ogni input (testuale o vocale) deve essere processato dal parser NLU per estrarre i dati strutturati.
 - *Estensione («extend»)*: è esteso da "**Conversione STT**" se la condizione `input == vocale` è soddisfatta.
- **Gestire Transazioni:** permette la modifica o l'eliminazione di movimenti esistenti.
 - Anche questo caso d'uso include la logica di parsing NLU ed è esteso dalla conversione vocale, permettendo all'utente di modificare i dati tramite comandi vocali.

Analisi e Configurazione

Riguarda le funzionalità di monitoraggio e organizzazione dei dati.

- **Consultare Stato Finanziario:** visualizzazione immediata del saldo attuale.
- **Gestire Budget:** definizione dei limiti di spesa mensili.
- **Gestire Categorie:** permette all'utente di creare, modificare o eliminare le categorie di spesa personalizzate (es. "Svago", "Casa").
- **Visualizzare Report:** generazione di un report mensili per l'utente.

3.5 Vista logica – Class Diagram

La vista logica (Figura 3.2) illustra la struttura statica del sistema, evidenziando la suddivisione in microservizi e le relazioni tra le classi. Il diagramma riflette l'architettura modulare adottata, dove ogni componente ha responsabilità ben definite.

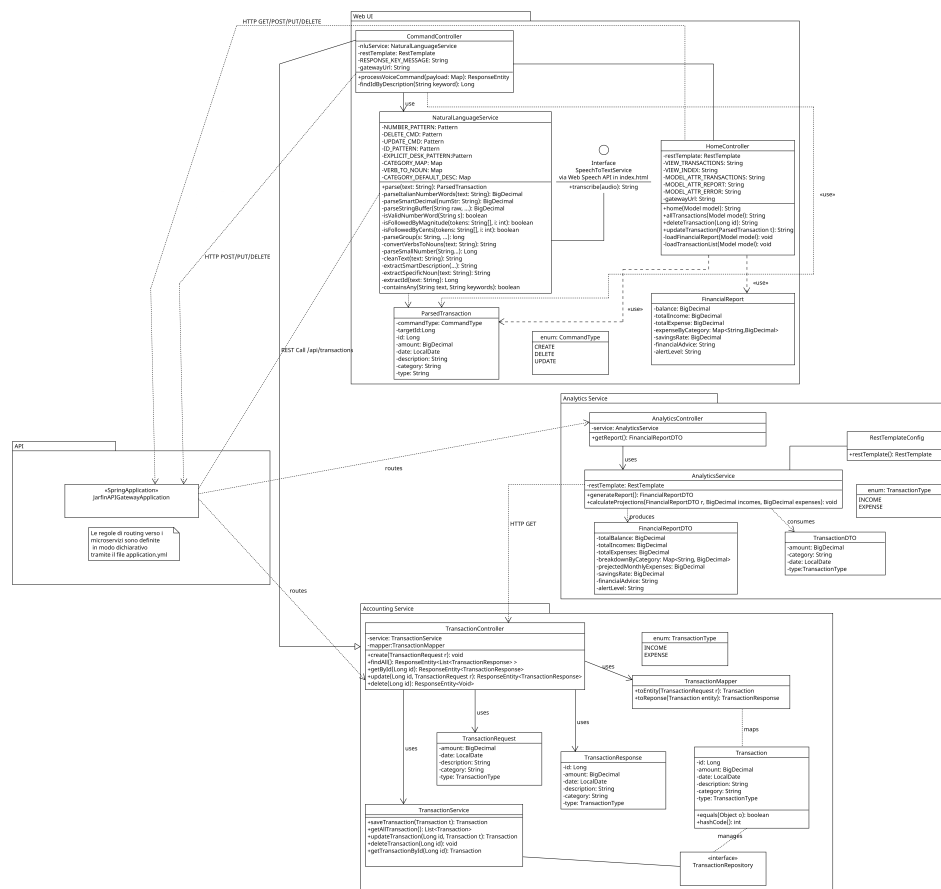


Figura 3.2: Diagramma di classi (Vista Logica).

3.5.1 Shared Kernel e Modello di Dominio

L'entità core del sistema è definita all'interno del servizio di Accounting:

- **Transaction:** Rappresenta l'entità fondamentale mappata sul database. Include i campi `id` (Long), `amount` (BigDecimal), `date` (LocalDate), `description` (String), `category` (String) e `type` (Enum INCOME/EXPENSE).

3.5.2 Web UI e NLU Service

Questo modulo gestisce l'interfaccia utente e l'interpretazione dei comandi vocali:

- **CommandController e HomeController:** Ricevono le richieste dal frontend. Il **CommandController** elabora i payload vocali invocando localmente il servizio NLU e successivamente usa il **RestTemplate** per instradare le richieste al Gateway.
- **NaturalLanguageService:** Il componente core che esegue il parsing del testo utilizzando pattern predefiniti e mappe di categorie per estrarre i dati transazionali.
- **ParsedTransaction:** Data Transfer Object (DTO) che racchiude l'intento estratto, includendo il **CommandType** (CREATE, DELETE, UPDATE) e i dati della transazione.

3.5.3 Analytics Service

Localizzato nel package dedicato, si occupa della generazione della reportistica finanziaria:

- **AnalyticsController:** Espone l'endpoint `getReport()` che restituisce un oggetto di tipo **FinancialReportDTO**.
- **AnalyticsService:** Contiene la logica per richiedere le transazioni all'Accounting Service (via **RestTemplate**) e aggregare i dati tramite il metodo `generateReport()`.
- **FinancialReportDTO:** Struttura dati ottimizzata per il frontend che include `totalBalance`, `totalIncomes`, `totalExpenses` e `breakdownByCategory`.

3.5.4 Accounting Service

Modulo preposto alla gestione della persistenza e della logica transazionale:

- **TransactionController**: Punto di ingresso per le operazioni CRUD (`create`, `findAll`, `getById`, `update`, `delete`).
- **TransactionService**: Implementa i metodi operativi come `saveTransaction()`, `updateTransaction()` e `deleteTransaction()`.
- **TransactionRepository**: Interfaccia Spring Data JPA per l'accesso diretto ai dati su PostgreSQL.

3.5.5 Orchestrazione e API Gateway

L'architettura utilizza un **JarfinApiGatewayApplication**:

- **Routing**: Funge da unico punto di accesso per le API di backend, instradando le richieste provenienti dalla Web UI verso i controller di **Accounting** e **Analytics**.

3.6 Vista di sviluppo – Component Diagram

Il Diagramma dei Componenti (Figura 3.3) illustra l'organizzazione del sistema JarFin in moduli software distinti e le dipendenze che intercorrono tra di essi. Questa vista evidenzia una struttura a tre livelli (*Layered Architecture*):

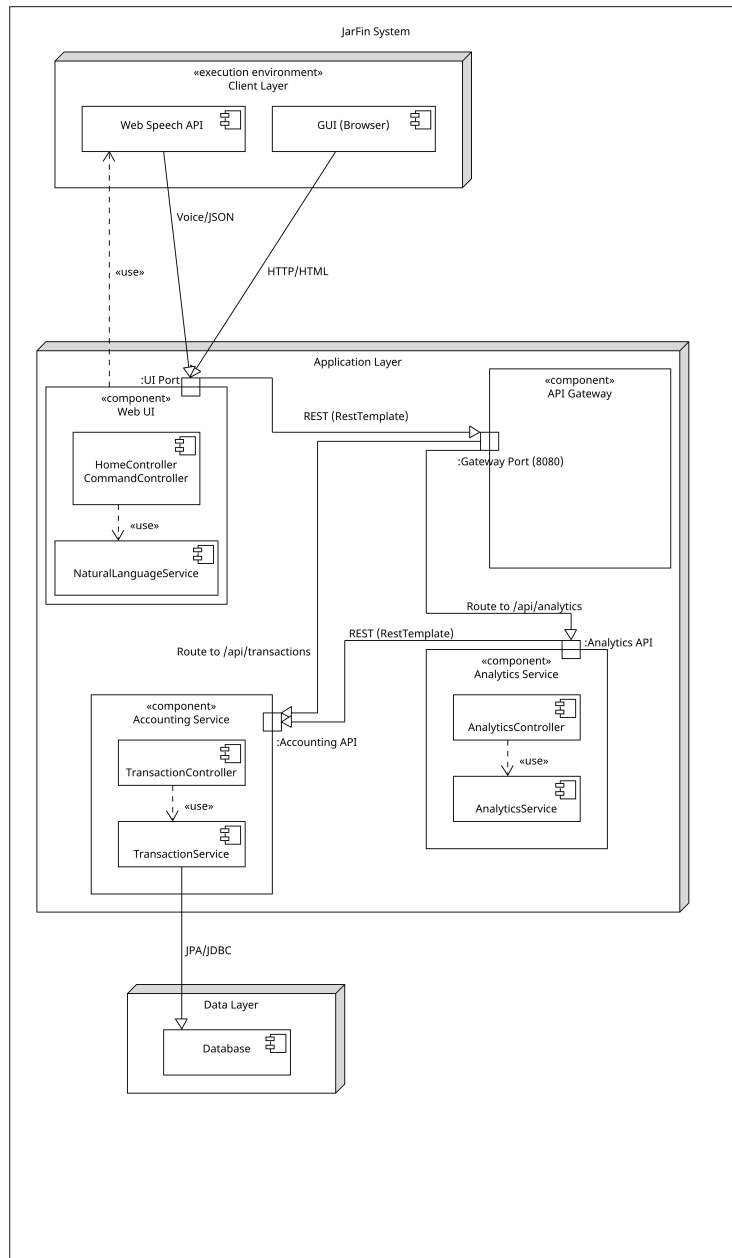


Figura 3.3: Diagramma dei componenti: architettura a livelli e interfacce esposte.

3.6.1 Client Layer

Il livello superiore rappresenta il frontend e i servizi di interazione immediata con l'utente:

- **GUI:** Componente che gestisce la logica di visualizzazione, implementando l'interfaccia `UserInterface` con i metodi `inputCommand()` e `displayReport()`.
- **Web Speech API (Browser):** Componente che fornisce le capacità di riconoscimento vocale integrate nel browser.
- **Porte:** La comunicazione verso i livelli sottostanti è veicolata tramite la `GUIPORT` [1].

3.6.2 Application Layer

Il nucleo del sistema incapsula la logica di business ed è suddiviso in quattro moduli principali (microservizi):

- **Web UI:** Contiene i controller web e incapsula il `NaturalLanguageService`. Funge da Backend For Frontend (BFF).
- **API Gateway:** Il punto di ingresso centralizzato che instrada il traffico verso i microservizi di dominio.
- **Accounting Service:** Gestisce i flussi finanziari e la logica CRUD esponendo le proprie API REST.
- **Analytics Service:** Aggrega i dati interrogando l'Accounting Service per calcolare le statistiche.

3.6.3 Data Layer

Il livello di persistenza è isolato dal resto dell'applicazione per garantire l'indipendenza dallo storage:

- **Database:** Componente responsabile dell'archiviazione dei dati.
- **DBInterface:** L'interfaccia di accesso ai dati che astrae le operazioni CRUD fondamentali: `save()`, `findAll()`, `findById()` e `deleteById()`.
- **Porte:** Riceve le richieste dal livello applicativo sulla porta `DBPORT` [1] attraverso una connessione verso la `DatabasePort`.

3.6.4 Dipendenze e Relazioni

- **Relazione di Uso:** Il componente Web UI presenta una dipendenza «use» verso la Web Speech API del Client Layer per le funzionalità STT.
- **Integrazione e Routing:** La Web UI comunica con l'API Gateway, che a sua volta instrada verso Accounting e Analytics. L'Analytics Service interroga l'Accounting Service per ottenere i dati necessari all'aggregazione.

3.7 Vista di processo – Sequence Diagram

La vista di processo descrive la dinamica temporale delle interazioni tra gli attori e i componenti del sistema JarFin durante l'esecuzione di un comando. Il diagramma (Figura 3.4) illustra il flusso completo, dalla ricezione dell'input alla persistenza dei dati e alla generazione del feedback.

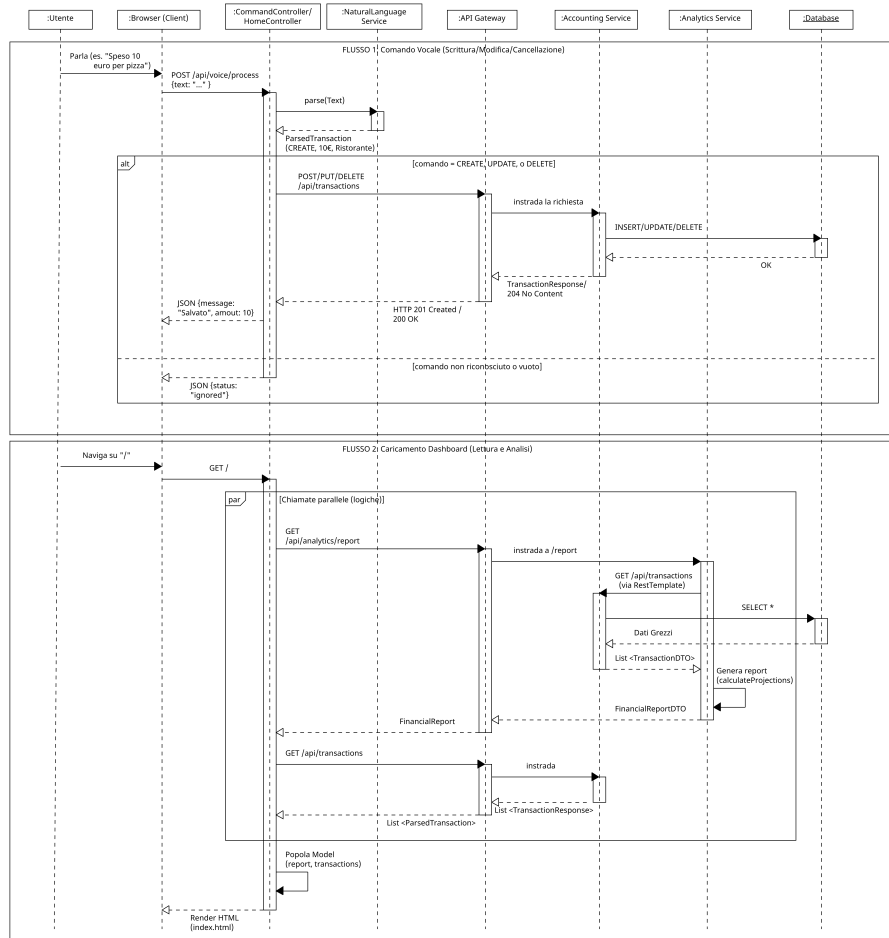


Figura 3.4: Diagramma di sequenza: autenticazione, gestione comandi e routing.

3.7.1 Fase 1: Input e Comando Vocale

Il flusso per l'inserimento o la modifica tramite voce ha inizio con l'interazione dell'utente:

1. **Invio Comando:** L'utente parla, il browser converte l'audio in testo e lo invia alla Web UI tramite `POST /api/voice/process`.
2. **Analisi Semantica:** Il Controller invoca localmente il `NaturalLanguageService` per estrarre l'intento e i dati.
3. **Inoltro al Gateway:** La Web UI inoltra la richiesta strutturata all'API Gateway, che la instrada all'**Accounting Service**.
4. **Persistenza:** L'Accounting Service esegue l'operazione (INSERT/UPDATE/-DELETE) sul Database, che restituisce conferma. L'esito risale fino alla Web UI.

3.7.2 Fase 2: Caricamento Dashboard (Lettura e Analisi)

Quando l'utente visita la pagina principale per consultare i report:

- La Web UI richiede il report all'API Gateway, che instrada la chiamata all'**Analytics Service**.
- L'**Analytics Service** effettua una chiamata REST all'**Accounting Service** per ottenere la lista delle transazioni correnti.
- L'Accounting Service interroga il Database (SELECT), ottiene i dati grezzi e li restituisce.
- Ricevuti i dati, l'Analytics calcola i totali e le proiezioni, restituendo il **FinancialReport** alla Web UI per il rendering HTML.

3.7.3 Fase 3: Feedback Finale

Una volta completata l'elaborazione nei microservizi, il ciclo si conclude con la notifica all'utente:

1. L'API Gateway invia la **Risposta Utente** alla Web UI.
2. L'applicazione mostra il **Feedback** finale (sotto forma di testo e interfacce aggiornate) all'utente.

3.8 Vista fisica – Deployment Diagram

Il Diagramma di Deployment (Figura 3.5) illustra la distribuzione fisica del sistema JarFin, mappando gli artefatti software sui nodi hardware e descrivendo i protocolli di comunicazione utilizzati.

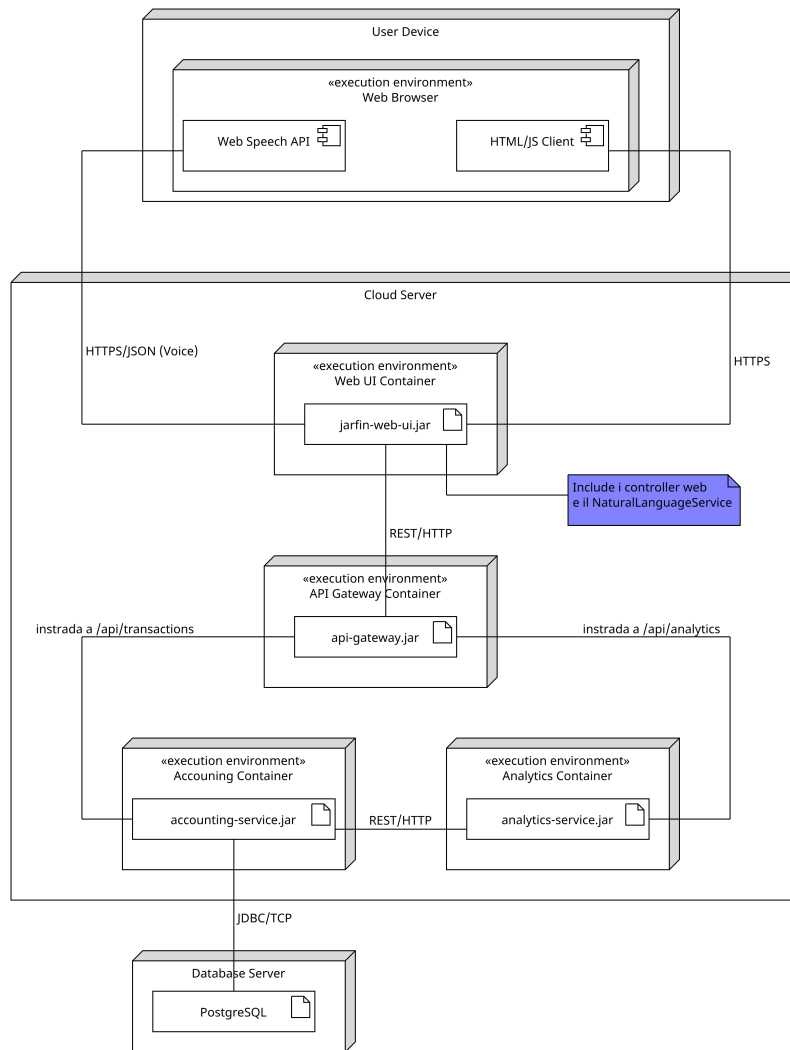


Figura 3.5: Diagramma di deployment: nodi fisici, container e protocolli di rete.

3.8.1 Architettura dei Nodi e Ambienti

Il sistema è distribuito su infrastrutture distinte che separano il frontend, la logica applicativa e la persistenza:

- **User Device:** Rappresenta il dispositivo terminale dell'utente. Ospita l'ambiente di esecuzione **Web Browser** con le interfacce client e sfrutta la **Web Speech API** per la conversione locale dell'audio in testo (STT).

- **Cloud Server:** Il nodo centrale che ospita il backend Java Spring Boot. È suddiviso in ambienti di esecuzione containerizzati per garantire l'isolamento:
 - **Web UI Container:** Esegue l'artefatto `jarfin-web-ui.jar` (che include la logica NLU e serve le pagine web).
 - **API Gateway Container:** Esegue l'artefatto `api-gateway.jar`.
 - **Accounting Container:** Esegue l'artefatto `accounting-service.jar`.
 - **Analytics Container:** Esegue l'artefatto `analytics-service.jar`.
- **Database Server:** Un nodo dedicato che ospita il DBMS **PostgreSQL**, garantendo che la persistenza dei dati sia fisicamente separata dalla logica di business per ottimizzare sicurezza e performance.

3.8.2 Protocolli di Comunicazione

Le interconnessioni tra i vari nodi utilizzano protocolli standard per lo scambio di dati:

- **User Device** $\xrightarrow{HTTPS/JSON}$ **Cloud Server:** La comunicazione tra il browser dell'utente e il server applicativo avviene tramite il protocollo **HTTPS**. I dati vengono scambiati in formato **JSON**, permettendo al client di inviare comandi e ricevere report o conferme.
- **Cloud Server** $\xrightarrow{JDBC}$ **Database Server:** L'accesso ai dati da parte dei micro-servizi (in particolare il servizio Accounting) avviene tramite una connessione **JDBC** verso l'istanza di PostgreSQL.
- **Integrazione Locale:** A differenza di implementazioni puramente cloud, il diagramma evidenzia che la componente di trascrizione vocale (**Web Speech API**) risiede nello *User Device*, riducendo la necessità di inviare flussi audio grezzi verso nodi esterni per il solo STT.

3.9 Conclusioni dell'Iterazione 0

L'attività di analisi e progettazione ha prodotto un'architettura solida, modulare e documentata. L'adozione del modello 4+1 ha permesso di affrontare la complessità del sistema distribuito, definendo chiaramente le responsabilità di ogni componente prima dell'inizio della fase di codifica (Iterazione 1).

Capitolo 4

Iterazione 1: Implementazione dell'Accounting Service

4.1 Introduzione e Obiettivi dell'Iterazione

L'Iterazione 1 del progetto *JarFin* ha rappresentato la fase fondativa dello sviluppo, dedicata alla progettazione e alla realizzazione del microservizio **Accounting Service**. Questo componente costituisce il nucleo operativo dell'intera architettura, essendo responsabile della gestione autoritativa delle transazioni finanziarie (registrazione, consultazione, modifica e cancellazione).

L'obiettivo primario di questa fase non si è limitato alla semplice implementazione delle funzionalità CRUD (*Create, Read, Update, Delete*), ma ha riguardato la costruzione di un'infrastruttura software solida, basata sui principi della *Clean Architecture* e pronta per supportare la scalabilità futura. Le sfide principali affrontate hanno riguardato:

- **Infrastruttura:** Configurazione di un ambiente di sviluppo containerizzato e riproducibile tramite Docker.
- **Persistenza:** Gestione rigorosa dei dati su un RDBMS relazionale (PostgreSQL).
- **Architettura:** Definizione di pattern per la validazione, la gestione centralizzata degli errori e il disaccoppiamento dei dati (DTO/Mapper).
- **Qualità:** Garanzia della robustezza del codice tramite una strategia di testing ibrida (Unitari e di Integrazione).

4.2 Stack Tecnologico e Configurazione

Il microservizio è stato sviluppato adottando uno stack tecnologico moderno, scelto per garantire robustezza e facilità di manutenzione:

- **Java 21:** Utilizzato come linguaggio di programmazione, sfruttando le ultime funzionalità LTS (*Long Term Support*) per massimizzare performance e sicurezza.
- **Spring Boot 3.2.0:** Framework di riferimento per lo sviluppo di microservizi in ambiente Java. Ha permesso di abbattere i tempi di configurazione (*Convention over Configuration*) fornendo un server web integrato (Tomcat) e la gestione automatica delle dipendenze.
- **Spring Data JPA / Hibernate:** Per la gestione della persistenza tramite ORM (*Object-Relational Mapping*), astruendo la complessità delle query SQL dirette.
- **PostgreSQL 16:** Scelto come RDBMS per la sua affidabilità, il supporto alle transazioni ACID e la capacità di gestire carichi di lavoro complessi.
- **Docker & Docker Compose:** Per l'orchestrazione dell'ambiente di sviluppo e la containerizzazione del database.
- **Lombok:** Libreria utilizzata per ridurre il codice "boilerplate" (getter, setter, costruttori), migliorando la leggibilità delle classi.
- **Maven:** Per la gestione del ciclo di vita del software e delle dipendenze, come definito nel file `pom.xml`.

4.3 Orchestrazione (Docker Compose)

Una delle prime decisioni architetturali è stata quella di disaccoppiare l'applicazione dall'installazione locale del database. Per garantire che l'ambiente di sviluppo fosse deterministico e identico per ogni sviluppatore, si è optato per la virtualizzazione tramite **Docker**.

Il file `docker-compose.yml` definisce il servizio di database e le sue configurazioni:

```
1 services:
2   db:
3     image: postgres:15
4     container_name: jarfin_db
5     environment:
6       POSTGRES_USER: ${DB_USER:-postgres}
7       POSTGRES_PASSWORD: ${DB_PASSWORD:-password_segreta}
8       POSTGRES_DB: jarfin_db
9     ports:
10      - "5432:5432"
11     volumes:
12      - postgres_data:/var/lib/postgresql/data
13
14 volumes:
15   postgres_data:
```

4.3.1 Analisi delle Scelte Implementative

- **Persistenza dei Dati:** L'uso di un volume nominato (`postgres_data`) garantisce la persistenza dei dati anche in caso di arresto o rimozione dei container, evitando la perdita di informazioni tra le sessioni di sviluppo.
- **Sicurezza delle Credenziali:** Le credenziali di accesso non sono rigidamente fissate nel codice. La configurazione supporta l'iniezione tramite variabili d'ambiente (`${DB_PASSWORD}`), con valori di default visibili solo per facilitare la revisione accademica. Questo approccio predispone il sistema per pipeline CI/CD sicure.
- **Networking:** Il database è esposto sulla porta 5432, permettendo connessioni sia dall'applicazione Spring Boot che da tool di amministrazione esterni (es. DBeaver/pgAdmin).

L'applicazione Spring Boot si connette al container in questione tramite il file `application.properties`, configurato per aggiornare automaticamente lo schema del database all'avvio (`spring.jpa.hibernate.ddl-auto=update`), accelerando notevolmente la fase di prototipazione.

4.4 Architettura Software e Design Patterns

Il microservizio implementa un'architettura a strati rigorosa (*Layered Architecture*) per garantire la separazione delle responsabilità (*Separation of Concerns*). Il flusso dei dati attraversa i seguenti layer, in ordine:

- **Controller (API)**: espone gli endpoint REST e gestisce le richieste HTTP;
- **Mapper / DTO**: converte i dati tra il modello di dominio e il contratto API;
- **Service (Business Logic)**: incapsula la logica applicativa;
- **Repository (Data Access)**: gestisce l'accesso e la persistenza dei dati.

4.4.1 Modellazione del Dominio: La scelta di BigDecimal

L'entità `Transaction` rappresenta il concetto centrale del dominio. Durante la modellazione, è stata posta particolare attenzione al tipo di dato per l'attributo `amount` (importo).

L'uso di tipi primitivi a virgola mobile come `double` o `float` è stato scartato a causa dell'imprecisione intrinseca nella rappresentazione binaria dei decimali (standard IEEE 754), che può portare a errori di arrotondamento inaccettabili in ambito finanziario. È stato quindi adottato `java.math.BigDecimal`, che garantisce una precisione arbitraria e operazioni aritmetiche esatte.

```
1 @Entity
2 @Table(name = "transactions")
3 @Getter @Setter @NoArgsConstructor
4 public class Transaction {
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @Column(nullable = false, precision = 19, scale = 4)
10    private BigDecimal amount; // Precisione finanziaria
11                                garantita
12
13    @Column(nullable = false)
14    private LocalDate date;
```

```

14
15     @Enumerated(EnumType.STRING)
16     @Column(nullable = false)
17     private TransactionType type; // INCOME o EXPENSE
18
19     // ... altri campi (description, category)
20 }

```

Inoltre, per evitare problemi noti con i proxy di Hibernate, i metodi `equals()` e `hashCode()` sono stati implementati manualmente basandosi strettamente sull'identificatore univoco (`id`) dell'entità.

4.4.2 Pattern DTO e Mapper

Per disaccoppiare il modello di persistenza (Entity) dal contratto esposto all'esterno (API), è stato implementato il pattern DTO (Data Transfer Object).

- **TransactionRequest:** Utilizzato per l'inserimento dati. Include annotazioni di Jakarta Validation (`@NotNull`, `@DecimalMin`, `@NotBlank`) per garantire che i dati in ingresso siano validi prima ancora di raggiungere la logica di business.
- **TransactionResponse:** Utilizzato per la risposta. Filtra eventuali dati tecnici non necessari al client.

La conversione tra Entity e DTO è centralizzata nella classe `TransactionMapper`, mantenendo il Controller e il Service puliti da logiche di trasformazione dei dati.

4.5 Logica di Business e Gestione Errori

4.5.1 Service Layer

La classe `TransactionService` incapsula la logica core. Un aspetto rilevante è l'uso dell'annotazione `@Transactional` sui metodi di modifica, che garantisce l'atomicità delle operazioni sul database: o l'operazione ha successo completamente, o viene eseguito un rollback. Inoltre, è stato introdotto un sistema di logging (`@Slf4j`) per tracciare le operazioni critiche, fondamentale per il debugging in un ambiente distribuito.

4.5.2 Gestione Centralizzata delle Eccezioni

Invece di disseminare blocchi `try-catch` nel codice, è stato implementato un Global Exception Handler utilizzando `@RestControllerAdvice`. Questo componente intercetta le eccezioni a livello globale e le trasforma in risposte JSON strutturate con i corretti codici di errore HTTP.

```
1 @RestControllerAdvice
2 public class GlobalExceptionHandler {
3
4     // Gestione 404 Not Found
5     @ExceptionHandler(EntityNotFoundException.class)
6     public ResponseEntity<String> handleNotFound(
7         EntityNotFoundException ex) {
8         return ResponseEntity.status(HttpStatus.NOT_FOUND).
9             body(ex.getMessage());
10    }
11
12    // Gestione 400 Bad Request (Errori di Validazione)
13    @ExceptionHandler(MethodArgumentNotValidException.class)
14    public ResponseEntity<Map<String, String>>
15        handleValidationErrors(
16            MethodArgumentNotValidException ex) {
17        Map<String, String> errors = new HashMap<>();
18        ex.getBindingResult().getFieldErrors().forEach(error
19            ->
20                errors.put(error.getField(), error.
21                    getDefaultMessage()));
22        return ResponseEntity.badRequest().body(errors);
23    }
24 }
```

Questo approccio migliora drasticamente l'usabilità dell'API: se un client invia un importo negativo, riceverà un errore 400 chiaro e dettagliato invece di un generico errore 500.

4.6 Esposizione API: TransactionController

Il `TransactionController` espone le funzionalità tramite endpoint RESTful standard. È stata posta particolare cura nel rispetto della semantica HTTP. Il metodo `create`, in particolare, implementa il livello 2 del modello di maturità di Richardson (HTTP Verbs), restituendo l'header `Location` con l'URI della risorsa appena creata.

```
1 @PostMapping
2 public ResponseEntity<TransactionResponse> create(@Valid
3     @RequestBody TransactionRequest request) {
4     Transaction entity = mapper.toEntity(request);
5     Transaction saved = service.saveTransaction(entity);
6
7     // Costruzione dinamica dell'URI della risorsa
8     URI location = ServletUriComponentsBuilder.
9         fromCurrentRequest()
10            .path("/{id}")
11            .buildAndExpand(saved.getId())
12            .toUri();
13
14     return ResponseEntity.created(location)
15        .body(mapper.toResponse(saved));
16 }
```

4.7 Qualità del Codice e Strategia di Testing

La validazione dell'Iterazione 1 non si è limitata alla sola verifica funzionale, ma ha seguito un percorso strutturato partendo dall'analisi statica del codice, passando per i test unitari con monitoraggio della copertura, fino ai test di integrazione manuali.

4.7.1 Analisi Statica e Refactoring (SonarLint)

Prima di procedere con il testing dinamico, è stata eseguita un'analisi statica del codice utilizzando **SonarLint**. Questa fase ha permesso di rilevare violazioni delle convenzioni REST e disuniformità nella gestione delle eccezioni.

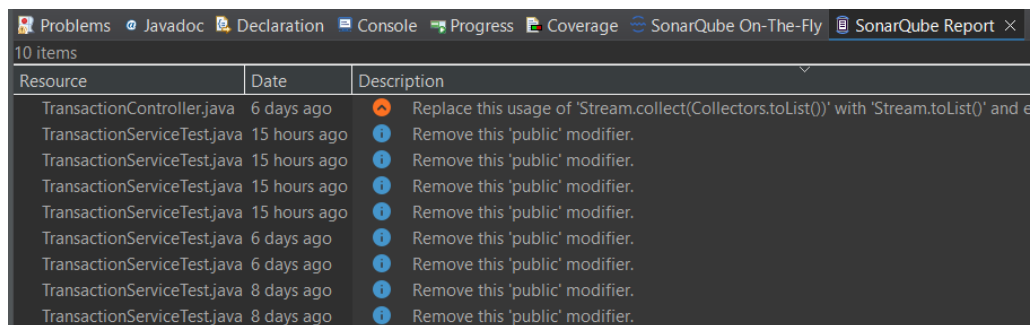
In particolare:

- **Problema rilevato:** Il `TransactionController` restituiva status code generici e mancava l'header `Location` nelle risposte POST, violando le best practices RESTful.
- **Intervento correttivo:** È stato introdotto `UriComponentsBuilder` ed eseguito un refactoring dei metodi per restituire oggetti `ResponseEntity` tipizzati correttamente, migliorando la semantica delle API.

```
35 ● @PostMapping
36 public ResponseEntity<TransactionResponse> create(@Valid @RequestBody TransactionRequest request) {
37     Transaction entity = mapper.toEntity(request);
38
39     Transaction saved = service.saveTransaction(entity);
40
41     URI location = ServletUriComponentsBuilder.fromCurrentRequest()
42         .path("/{id}")
43         .buildAndExpand(saved.getId())
44         .toUri();
45
46     return ResponseEntity.created(location)
47         .body(mapper.toResponse(saved));
48 }
```

Figura 4.1: Refactoring del Controller per conformità REST

Inoltre, sono state applicate correzioni minori suggerite dal linter, come la rimozione del modificatore `public` nei metodi di test (Figura 4.2), rendendo il codice più pulito e conforme alle convenzioni di JUnit 5.



Resource	Date	Description
TransactionController.java	6 days ago	Replace this usage of 'Stream.collect(Collectors.toList())' with 'Stream.toList()' and e
TransactionServiceTest.java	15 hours ago	Remove this 'public' modifier.
TransactionServiceTest.java	15 hours ago	Remove this 'public' modifier.
TransactionServiceTest.java	15 hours ago	Remove this 'public' modifier.
TransactionServiceTest.java	15 hours ago	Remove this 'public' modifier.
TransactionServiceTest.java	6 days ago	Remove this 'public' modifier.
TransactionServiceTest.java	6 days ago	Remove this 'public' modifier.
TransactionServiceTest.java	8 days ago	Remove this 'public' modifier.
TransactionServiceTest.java	8 days ago	Remove this 'public' modifier.

Figura 4.2: Esempio di code smell rilevato da SonarLint

4.7.2 Unit Testing e Copertura (JUnit 5, Mockito & EclEmma)

Per la verifica della logica di business, sono stati sviluppati test unitari isolati per il Service e il Controller. L'uso di **Mockito** ha permesso di simulare il comportamento del database (pattern *mocking*), verificando la corretta generazione delle risposte HTTP e l'elaborazione dei dati senza dipendenze esterne.

Strategia di Copertura

Non è stato perseguito l'obiettivo formale del 100% di copertura, ma si è adottato un approccio **mirato** (soglia minima 85%), concentrandosi sui metodi critici che:

- contengono logica di business non banale o calcoli finanziari;
- garantiscono l'integrità dei dati (es. validazione importi negativi);
- gestiscono la persistenza e le transazioni verso il database.

La Figura 4.3 mostra l'esecuzione con successo della suite di test nell'ambiente di sviluppo.

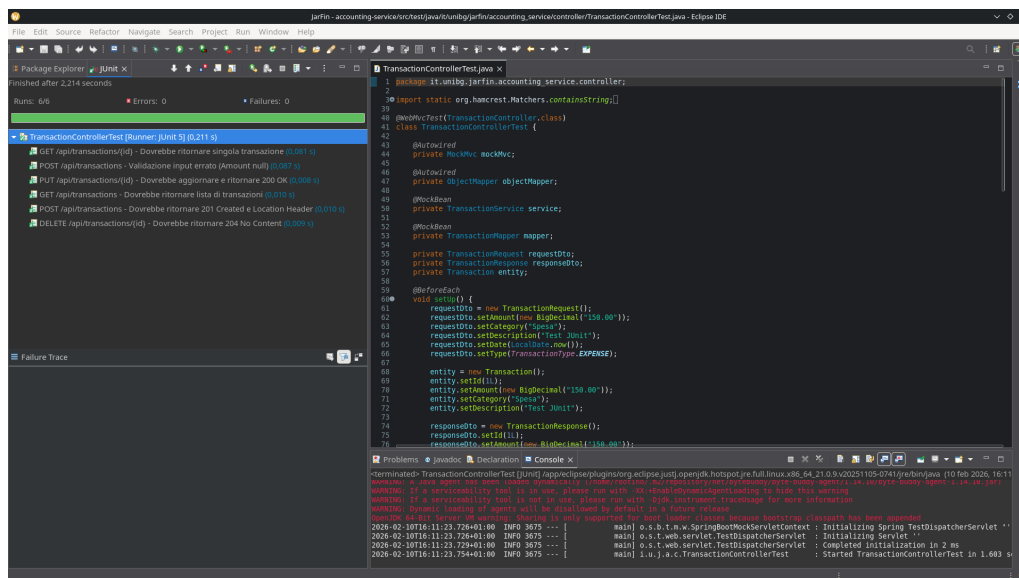


Figura 4.3: Suite di test unitari completata con successo su TransactionController

Metriche Raggiunte

L'analisi con **EclEmma** ha confermato l'efficacia della strategia, raggiungendo livelli di copertura eccellenti per il modulo di contabilità:

- **Service Layer Coverage: 100%**
- **Controller Layer Coverage: 100%**
- **Accounting Service Package Coverage: 96%**

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
accounting-service	96,0 %	1.380	57	1.437
src/main/java	89,0 %	314	39	353
it.unibg.jarfin.accounting_service	37,5 %	3	5	8
AccountingServiceApplication.java	37,5 %	3	5	8
it.unibg.jarfin.accounting_service.controller	100,0 %	84	0	84
TransactionController.java	100,0 %	84	0	84
it.unibg.jarfin.accounting_service.exception	100,0 %	34	0	34
EntityNotFoundException.java	100,0 %	4	0	4
GlobalExceptionHandler.java	100,0 %	30	0	30
it.unibg.jarfin.accounting_service.mapper	100,0 %	59	0	59
TransactionMapper.java	100,0 %	59	0	59
it.unibg.jarfin.accounting_service.model	41,4 %	24	34	58
Transaction.java	0,0 %	0	34	34
TransactionType.java	100,0 %	24	0	24
it.unibg.jarfin.accounting_service.service	100,0 %	110	0	110
TransactionService.java	100,0 %	110	0	110

Figura 4.4: Report di copertura EclEmma per il microservizio accounting-service

4.7.3 Analisi Architeturale (CodeMR)

Per valutare la qualità strutturale del microservizio, è stato utilizzato **CodeMR**. L'analisi ha confermato una costruzione ottima delle classi, evidenziando un basso accoppiamento e un'alta coesione.

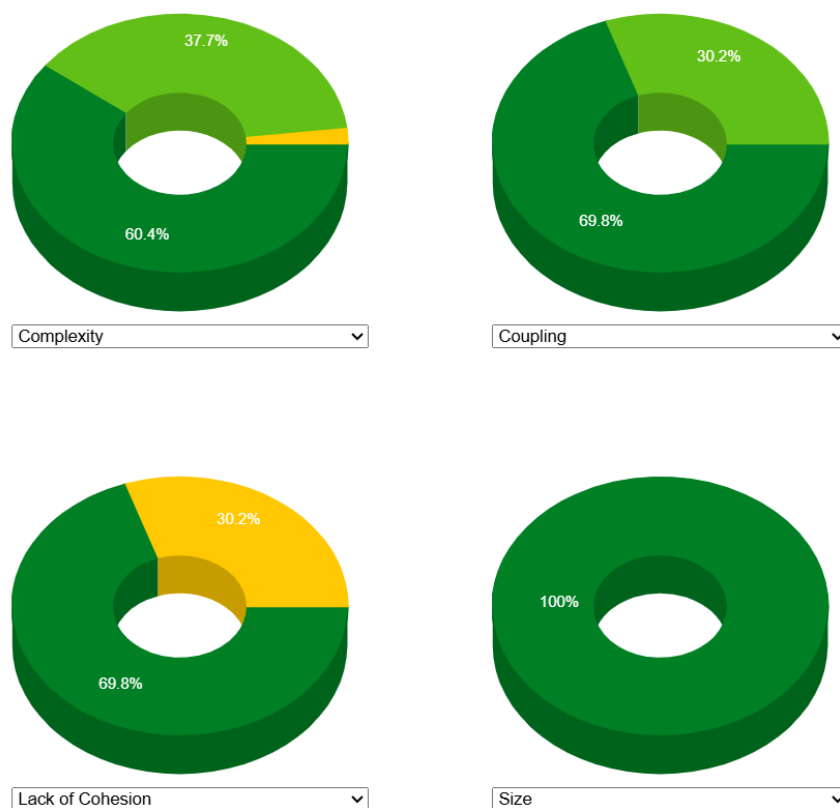


Figura 4.5: Metriche degli attributi di qualità rilevate

Il grafico delle dipendenze (Figura 4.6) offre una rappresentazione visiva delle relazioni tra componenti, confermando una corretta stratificazione tra Controller, Service e Repository.

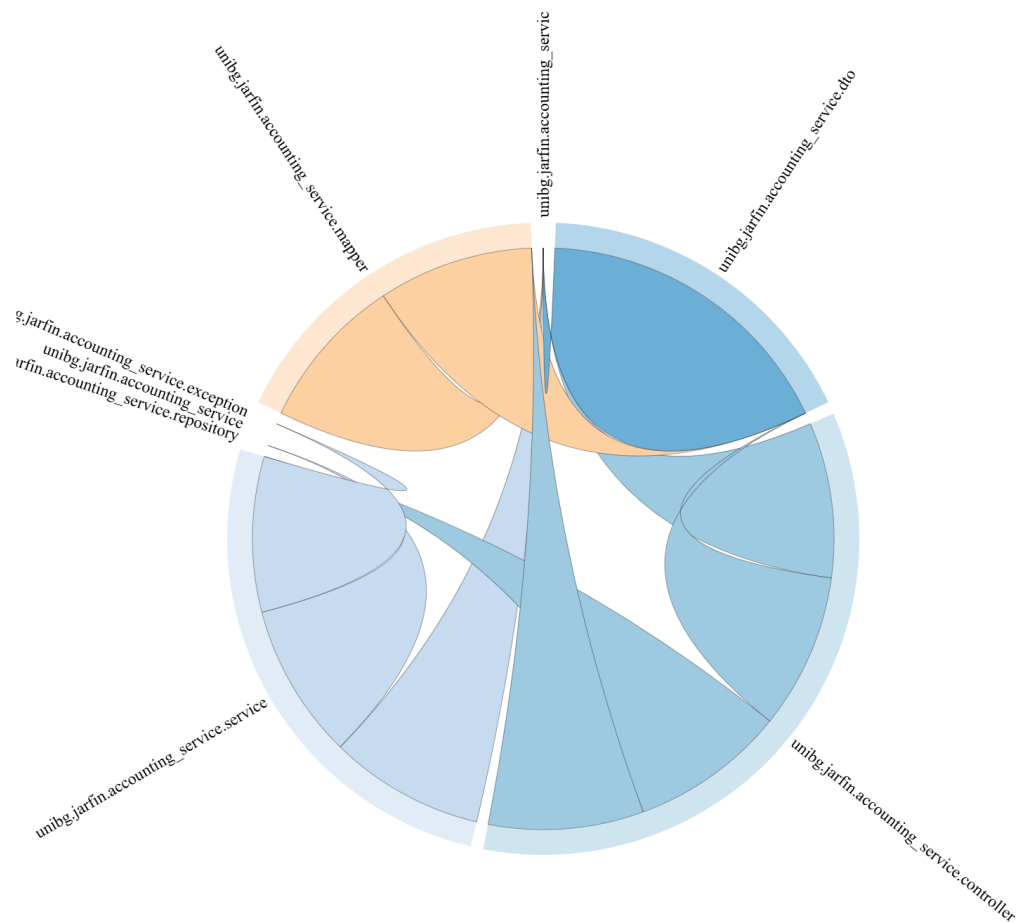


Figura 4.6: Grafo delle dipendenze del package Accounting

4.7.4 Integration Testing (Postman)

A completamento della fase di test, è stata eseguita una validazione *End-to-End* dell'intero stack (Controller + Service + DB + Docker) mediante chiamate REST manuali con **Postman**.

1. Creazione Risorsa (Happy Path) Una richiesta POST valida (Figura 4.7) restituisce status 201 **Created** e il JSON della transazione salvata, confermando il corretto funzionamento della catena di persistenza.

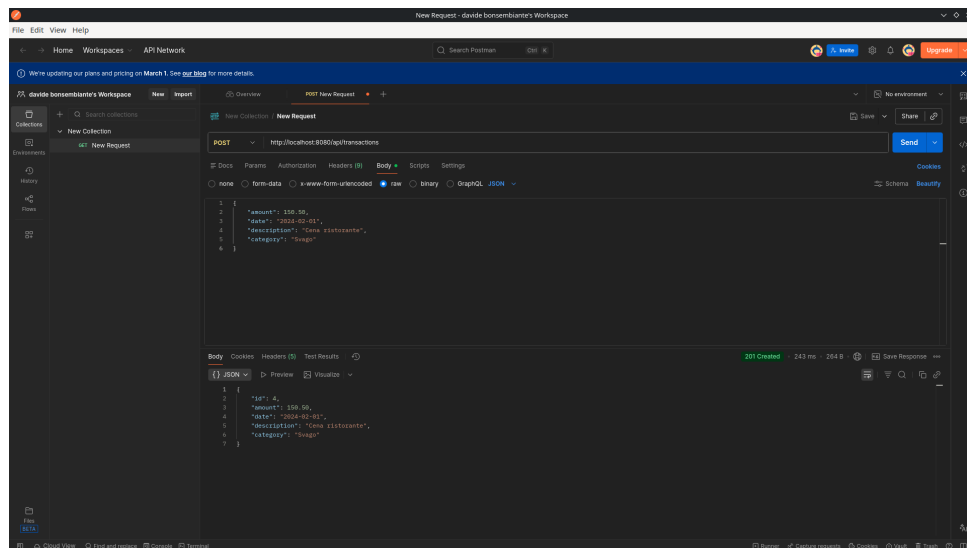


Figura 4.7: Creazione di una transazione con successo (Status 201).

2. Recupero Dati La richiesta GET (Figura 4.8) conferma che i dati sono stati persistiti correttamente sul database PostgreSQL containerizzato.

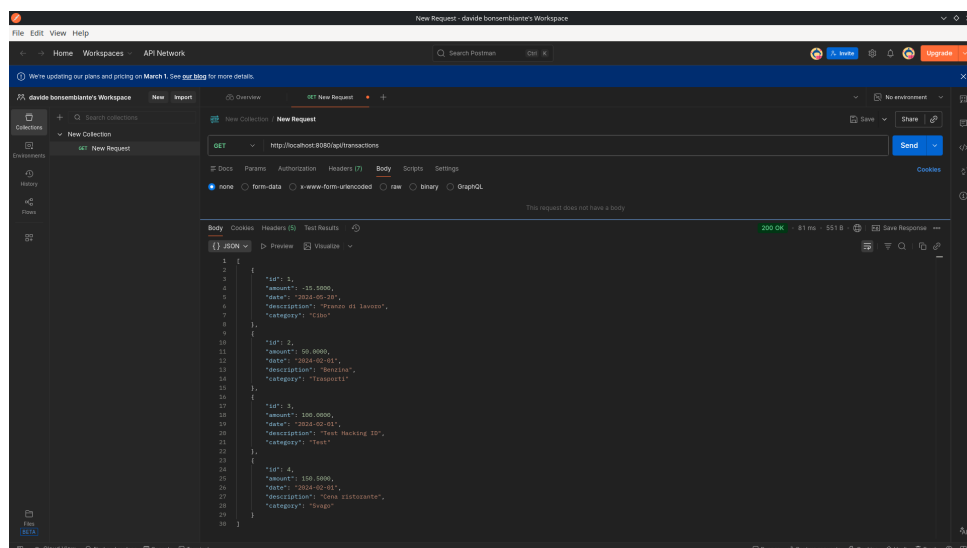


Figura 4.8: Recupero della lista delle transazioni (Status 200).

3. Validazione Input (Error Handling) Inviando un payload con dati non validi (es. importo negativo), il sistema attiva il `GlobalExceptionHandler` (Figura 4.9), restituendo un `400 Bad Request` con i dettagli dell'errore.

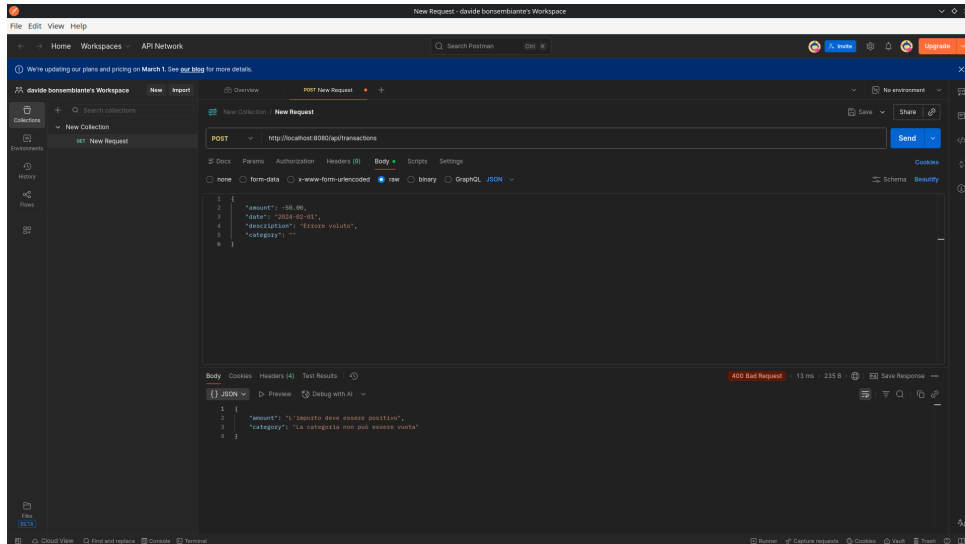


Figura 4.9: Test di validazione: risposta 400 Bad Request su importo negativo.

4.8 Conclusione Iterazione 1

L'Iterazione 1 ha consegnato un microservizio di Accounting pienamente operativo, validato sia staticamente che dinamicamente. L'integrazione di strumenti come SonarLint e CodeMR ha garantito la qualità del codice fin dalle prime fasi, mentre l'alta copertura dei test unitari (96%) assicura la robustezza della logica di business. Il sistema, containerizzato con Docker e validato via Postman, è ora una base solida pronta per l'integrazione con i moduli di Analytics e Natural Language Understanding.

Capitolo 5

Iterazione 2: Implementazione dell'Analytics Service

5.1 Obiettivi e Ruolo Architettuale

L'Iterazione 2 sposta il focus dalla gestione del dato (persistenza) alla sua elaborazione (intelligenza). L'obiettivo è la realizzazione dell'**Analytics Service**, un microservizio stateless progettato per utilizzare i dati grezzi forniti dall'Accounting Service e trasformarli in metriche finanziarie di alto livello.

A differenza del servizio precedente, qui la complessità non risiede nell'interazione con il database (assente in questo modulo), ma nella logica algoritmica di aggregazione e proiezione. Le sfide principali affrontate sono state:

- L'implementazione di una comunicazione sincrona tra microservizi robusta e configurabile.
- La progettazione di algoritmi di aggregazione efficienti con complessità lineare $O(n)$.
- La garanzia della correttezza matematica dei calcoli finanziari tramite test isolati.

5.2 Stack Tecnologico e Configurazione

L'applicazione condivide lo stack base (Java 21, Spring Boot 3.2.0, Lombok) ma introduce componenti specifici per il suo ruolo di "Consumer".

5.2.1 Configurazione del Client HTTP (RestTemplate)

Per comunicare con l'Accounting Service, è stato scelto **RestTemplate**. Per evitare di istanziare manualmente l'oggetto in ogni classe (anti-pattern), è stato configurato come un Bean gestito dal container di Spring.

```
1 @Configuration
2 public class RestTemplateConfig {
3     @Bean
4     public RestTemplate restTemplate() {
5         return new RestTemplate();
6     }
7 }
```

Questa scelta permette, in futuro, di iniettare configurazioni globali come timeout di connessione o intercettori per l'autenticazione, centralizzando la gestione della rete.

5.2.2 Externalizzazione delle Dipendenze

Un aspetto critico nei sistemi distribuiti è evitare di "scrivere nel codice" gli indirizzi IP o gli URL degli altri servizi (*Hardcoding*). Nel file `application.properties`, l'URL del servizio produttore è parametrizzato:

```
1 server.port=8082
2 spring.application.name=analytics-service
3
4 # URL con valore di default, sovrascrivibile da variabili d'
   ambiente
5 accounting.service.url=${ACCOUNTING_SERVICE_URL:http://
   localhost:8081/api/transactions}
```

Questo approccio rispetta il principio III delle *12-Factor App* (Config), permettendo di cambiare l'indirizzo del servizio Accounting (es. in un ambiente Docker o Kubernetes) senza ricompilare il jar.

5.3 Implementazione della Logica di Business

Il cuore del servizio è la classe `AnalyticsService`. Analizziamo le decisioni implementative riga per riga.

5.3.1 Modellazione dei Dati (Type Safety)

Prima di eseguire calcoli, è stato necessario definire un contratto dati rigoroso. L'uso di `BigDecimal` è stato confermato per tutti i campi monetari. Inoltre, per gestire la distinzione tra entrate e uscite, si è evitato l'uso di convenzioni fragili (come il segno meno) a favore di un Enum esplicito.

```
1 public enum TransactionType {  
2     INCOME,    // Entrata esplicita  
3     EXPENSE    // Uscita esplicita  
4 }
```

5.3.2 Algoritmo di Aggregazione (Design in Piccolo)

Il metodo `generateReport()` implementa l'algoritmo principale. Il suo compito è trasformare una lista piatta di transazioni in un oggetto strutturato `FinancialReportDTO`.

Fase 1: Fetching dei Dati Il servizio esegue una chiamata HTTP GET. L'array JSON ricevuto viene automaticamente deserializzato in un array di oggetti `TransactionDTO`.

```
1 public FinancialReportDTO generateReport() {  
2     TransactionDTO[] response = restTemplate.getForObject(  
3         accountingUrl, TransactionDTO[].class);  
4  
5     if (response == null || response.length == 0) {  
6         return new FinancialReportDTO(); // Oggetto vuoto (  
7             Pattern Null Object)  
8     }  
9     // ...  
}
```

Fase 2: Scansione e Accumulo L'algoritmo itera sulla lista una sola volta. Per ogni transazione, verifica il tipo e aggiorna i contatori. Contestualmente, popola una HashMap per raggruppare le spese per categoria.

```
1 BigDecimal incomes = BigDecimal.ZERO;
2 BigDecimal expenses = BigDecimal.ZERO;
3 Map<String, BigDecimal> categories = new HashMap<>();
4
5 for (TransactionDTO t : transactions) {
6     // Accumulo condizionale
7     if (t.getType() == TransactionType.INCOME) {
8         incomes = incomes.add(t.getAmount());
9     } else if (t.getType() == TransactionType.EXPENSE) {
10        expenses = expenses.add(t.getAmount());
11
12        // Aggregazione per categoria:
13        // Se la chiave esiste somma, altrimenti inizializza
14        categories.merge(t.getCategory(), t.getAmount(),
15                          BigDecimal::add);
16    }
17 }
```

La scelta di usare `Map.merge` è stilisticamente e performantemente superiore a un blocco `if-contains-put`, rendendo il codice più conciso.

5.4 Analisi dell'Algoritmo di Analytics

Il cuore dell'Analytics Service è l'algoritmo di elaborazione finanziaria implementato nel metodo `generateReport`. Questo algoritmo non si limita a una semplice somma, ma esegue una pipeline di trasformazione dei dati divisa in due stadi logici:

1. **Fase di Aggregazione (Iterativa):** Scansione dello storico transazionale per il calcolo dei totali e il raggruppamento per categorie.
2. **Fase di Proiezione e Valutazione (Analitica):** Applicazione di modelli predittivi e logiche decisionali basate su soglie (Alert System).

5.4.1 Pseudocodice Formale

Di seguito viene presentato lo pseudocodice completo dell'algoritmo, che evidenzia la complessità della logica di business implementata.

```
1  Algoritmo GenerateFinancialReport(transactions[]):
2      Input: Array  $T$  di  $N$  transazioni
3      Output: Report  $R$  con metriche e proiezioni
4
5      // -- FASE 1: Aggregazione Lineare --
6      Inizializza  $Sum_{inc} \leftarrow 0$ ,  $Sum_{exp} \leftarrow 0$ 
7      Inizializza  $Map_{cat} \leftarrow \emptyset$ 
8
9      FOR each  $t \in T$  DO:
10         IF  $t.type$  is INCOME THEN
11              $Sum_{inc} \leftarrow Sum_{inc} + t.amount$ 
12         ELSE IF  $t.type$  is EXPENSE THEN
13              $Sum_{exp} \leftarrow Sum_{exp} + t.amount$ 
14             // Gestione collisioni e merge
15              $Map_{cat}.merge(t.category, t.amount, SUM)$ 
16         END IF
17     END FOR
18
19     // -- FASE 2: Proiezioni e Valutazione (
20         calculateProjections) --
21      $D_{cur} \leftarrow$  Giorno corrente del mese
22      $D_{tot} \leftarrow$  Giorni totali nel mese corrente
23
24     // Proiezione Spese (Linear Projection Model)
25     IF  $D_{cur} > 0$  THEN
26          $Avg_{daily} \leftarrow Sum_{exp} / D_{cur}$ 
27          $Proj_{month} \leftarrow Avg_{daily} \times D_{tot}$ 
28     END IF
29
30     // Valutazione Salute Finanziaria (Decision Tree)
31      $Balance \leftarrow Sum_{inc} - Sum_{exp}$ 
32      $SavingsRate \leftarrow 0$ 
33      $Alert \leftarrow GRAY$ 
34
35     IF  $Sum_{inc} > 0$  THEN
```

```

35      SavingsRate  $\leftarrow$  (Balance/Suminc)  $\times$  100
36
37      IF SavingsRate > 20 THEN Alert  $\leftarrow$  GREEN
38      ELSE IF SavingsRate > 0 THEN Alert  $\leftarrow$  YELLOW
39      ELSE Alert  $\leftarrow$  RED
40  ELSE
41      // Edge Case: Spese senza entrate
42      IF Sumexp > 0 THEN Alert  $\leftarrow$  RED_CRITICAL
43  END IF
44
45  RETURN Report(Suminc, Sumexp, Mapcat, Projmonth, Alert)

```

5.4.2 Analisi della Complessità Computazionale

Per dimostrare l'efficienza e la scalabilità della soluzione, analizziamo il costo computazionale $T(n)$ in funzione del numero di transazioni n .

Il tempo totale di esecuzione è dato dalla somma dei costi delle due fasi:

$$T(n) = T_{agg}(n) + T_{proj}(n)$$

1. Costo della Fase di Aggregazione $T_{agg}(n)$

Questa fase richiede l'iterazione su tutte le n transazioni recuperate. All'interno del ciclo, vengono eseguite le seguenti operazioni elementari:

- **Type Checking:** Verifica del tipo di transazione ($O(1)$).
- **Aritmetica BigDecimal:** Le operazioni `add` su oggetti `BigDecimal` sono più costose delle primitive, dipendendo dalla precisione (numero di cifre significative P). Tuttavia, essendo la precisione fissata a 4 cifre decimali, possiamo considerare il costo costante k_{math} .
- **Aggiornamento HashMap:** L'operazione `merge` implica il calcolo dell'hash della categoria e l'accesso al bucket.
 - Caso Medio: $O(1)$ (distribuzione uniforme delle chiavi).
 - Caso Pessimo: $O(n)$ (tutte le chiavi collidono nello stesso bucket, degenerando in una lista linkata).

Assumendo una buona funzione di hash, il costo della fase 1 è:

$$T_{agg}(n) = \sum_{i=1}^n (c_{check} + k_{math} + c_{map}) \approx n \cdot C_1$$

$$T_{agg}(n) = O(n)$$

2. Costo della Fase di Proiezione $T_{proj}(n)$

Questa fase (metodo `calculateProjections`) non dipende dal numero di transazioni n , in quanto opera sui dati aggregati (Sum_{inc}, Sum_{exp}). Esegue un numero fisso di operazioni matematiche complesse (divisioni con arrotondamento, moltiplicazioni) e valutazioni condizionali (IF/ELSE).

$$T_{proj}(n) = C_2 \quad (\text{Costante})$$

$$T_{proj}(n) = O(1)$$

Complessità Totale

Combinando i risultati:

$$T(n) = O(n) + O(1) \implies T(n) = O(n)$$

L'algoritmo presenta una **complessità lineare**. Questo è un risultato ottimale per un sistema di aggregazione, garantendo che il tempo di risposta cresca proporzionalmente al volume dei dati senza esplosioni esponenziali.

5.4.3 Analisi della Complessità Spaziale

Lo spazio di memoria richiesto $S(n)$ dipende principalmente dalla struttura dati utilizzata per il raggruppamento delle categorie. Sia K il numero di categorie uniche presenti nelle transazioni.

$$S(n) = O(K)$$

Nel caso pessimo, in cui ogni transazione ha una categoria diversa, $K = n$, quindi $S(n) = O(n)$. Tuttavia, nel dominio reale, il numero di categorie è limitato e molto inferiore al numero di transazioni ($K \ll n$), rendendo l'algoritmo efficiente anche in termini di memoria.

5.5 Algoritmi Predittivi e Sistema Esperto

Oltre all'aggregazione, il servizio applica logiche di Business Intelligence.

5.5.1 Proiezione Temporale (Stima a fine mese)

Per calcolare la stima delle spese (`projectedMonthlyExpenses`), non è stata utilizzata una costante fissa (es. 30 giorni), ma la lunghezza esatta del mese corrente fornita da `java.time.LocalDate`.

```
1 LocalDate now = LocalDate.now();
2 int daysInMonth = now.lengthOfMonth(); // Es. 28, 30 o 31
3 int currentDay = now.getDayOfMonth();
4
5 BigDecimal dailyAverage = expenses.divide(
6     BigDecimal.valueOf(currentDay), 2, RoundingMode.HALF_UP);
7
8 BigDecimal projection = dailyAverage.multiply(
9     BigDecimal.valueOf(daysInMonth));
```

Questa attenzione al dettaglio ("Design for Accuracy") evita distorsioni nelle stime durante mesi particolari come Febbraio.

5.5.2 Alert System (Savings Rate)

Il calcolo del livello di allerta implementa una logica a soglie. Il codice gestisce anche i casi limite (divisione per zero se non ci sono entrate).

```
1 if (incomes.compareTo(BigDecimal.ZERO) > 0) {
2     // Calcolo percentuale risparmio
3     BigDecimal rate = balance.divide(incomes, 4, RoundingMode
4         .HALF_UP)
5         .multiply(new BigDecimal("100"));
6
7     if (rate.compareTo(new BigDecimal("20")) > 0) {
8         report.setAlertLevel("GREEN- Ottimo risparmio");
9         report.setFinancialAdvice("Ottimo lavoro! Continua a
10             risparmiare così.");
```

```

9      } else if (rate.compareTo(BigDecimal.ZERO) > 0) {
10          report.setAlertLevel("YELLOW-Attenzione alle spese"
11                                );
12          report.setFinancialAdvice("Cerca di ridurre le spese non essenziali.");
13      } else {
14          report.setAlertLevel("RED-Bilancio in negativo!");
15          report.setFinancialAdvice("Attenzione: le uscite superano le entrate.");
16      }
17  } else {
18      // Edge Case: Spese presenti ma zero entrate
19      if (expenses.compareTo(BigDecimal.ZERO) > 0) {
20          report.setAlertLevel("RED-Critical");
21          report.setFinancialAdvice("Attenzione! Stai spendendo senza alcuna entrata registrata.");
22      }
23  }

```

5.6 Qualità del Codice e Strategia di Testing

Nell'Iterazione 2, il focus si è spostato sulla validazione di logiche di business complesse (aggregazioni, proiezioni) e sulla gestione delle dipendenze esterne. Essendo l'Analytics Service un consumatore di dati (tramite chiamate HTTP verso l'Accounting Service), la strategia di testing ha dovuto isolare queste dipendenze per garantire determinismo e affidabilità.

5.6.1 Analisi della Complessità e Refactoring

L'analisi statica iniziale ha evidenziato che il metodo principale `generateReport` presentava un'elevata **complessità cognitiva** a causa di logiche annidate per il calcolo delle proiezioni finanziarie.

Per migliorare la manutenibilità, sono stati applicati i seguenti interventi di refactoring:

- **Estrazione della Logica:** Il calcolo delle proiezioni è stato isolato in metodi privati dedicati (es. `calculateProjections()`), riducendo la complessità ciclomatica del metodo principale.

- **Gestione delle Costanti:** Gli URL dei servizi e i parametri di soglia (es. "Magic Numbers" per gli alert) sono stati sostituiti con costanti `static final`, rendendo il sistema configurabile e meno soggetto a errori di battitura (Typos).

```

65 private void calculateProjections(FinancialReportDTO report, BigDecimal incomes, BigDecimal expenses) {
66     LocalDate today = LocalDate.now();
67     int dayOfMonth = today.getDayOfMonth();
68     int daysInMonth = today.lengthOfMonth();
69
70     if (dayOfMonth > 0) {
71         BigDecimal dailyAverage = expenses.divide(BigDecimal.valueOf(dayOfMonth), 2, RoundingMode.HALF_UP);
72         BigDecimal projected = dailyAverage.multiply(BigDecimal.valueOf(daysInMonth));
73         report.setProjectedMonthlyExpenses(projected);
74     }
75
76     if (incomes.compareTo(BigDecimal.ZERO) > 0) {
77         BigDecimal balance = incomes.subtract(expenses);
78         BigDecimal rate = balance.divide(incomes, 4, RoundingMode.HALF_UP)
79             .multiply(BigDecimal.valueOf(100));
80         report.setSavingsRate(rate);
81
82         if (rate.compareTo(new BigDecimal("20")) > 0) {
83             report.setAlertLevel("GREEN - Ottimo risparmio");
84             report.setFinancialAdvice("Ottimo lavoro! Continua a risparmiare così.");
85         } else if (rate.compareTo(BigDecimal.ZERO) > 0) {
86             report.setAlertLevel("YELLOW - Attenzione alle spese");
87             report.setFinancialAdvice("Cerca di ridurre le spese non essenziali.");
88         } else {
89             report.setAlertLevel("RED - Bilancio in negativo!");
90             report.setFinancialAdvice("Attenzione: le uscite superano le entrate.");
91         }
92     } else {
93         if (expenses.compareTo(BigDecimal.ZERO) > 0) {
94             report.setSavingsRate(new BigDecimal("-100"));
95             report.setAlertLevel("RED - Critical");
96             report.setFinancialAdvice("⚠ Attenzione! Stai spendendo senza alcuna entrata registrata.");
97         } else {
98             report.setSavingsRate(BigDecimal.ZERO);
99             report.setAlertLevel("GRAY - Nessun dato");
100             report.setFinancialAdvice("Registra delle transazioni per iniziare l'analisi.");
101         }
102     }
103 }

```

Figura 5.1: Estrazione del metodo `calculateProjections` per ridurre la complessità

5.7 Testing Isolato con Mockito

Data la natura interconnessa del servizio, un semplice Unit Test non sarebbe sufficiente né affidabile se richiedesse l'istanza reale dell'Accounting Service attiva. Per garantire che i test siano deterministici e veloci, è stato utilizzato il framework **Mockito**.

La classe di test `AnalyticsServiceTest` utilizza le annotazioni:

- `@Mock`: Crea una versione simulata di `RestTemplate`.
- `@InjectMocks`: Inietta il mock dentro l'istanza reale di `AnalyticsService`.

I test verificano la correttezza matematica dell'aggregatore e la logica degli alert (GREEN, RED, CRITICAL) fornendo dati fittizi tramite la tecnica dello "Stubbing".

Di seguito è riportato lo scenario "GREEN" (Risparmio positivo), dove si verifica che il calcolo del *Savings Rate* e del *Total Balance* sia corretto:


```

1  @Test
2  @DisplayName("Scenario GREEN: Entrate > Uscite (Risparmio Alto)")
3  void generateReport_GreenScenario() {
4      // ARRANGE: Preparazione dati fittizi
5      TransactionDTO income = new TransactionDTO();
6      income.setAmount(new BigDecimal("2000.00"));
7      income.setType(TransactionType.INCOME);
8
9      TransactionDTO expense = new TransactionDTO();
10     expense.setAmount(new BigDecimal("500.00"));
11     expense.setType(TransactionType.EXPENSE);
12     expense.setCategory("Svago");
13
14     TransactionDTO[] mockResponse = {income, expense};
15
16     // Istruiamo il Mock a restituire questi dati quando
17     // chiamato
18     when(restTemplate.getForObject(any(), eq(TransactionDTO
19     [].class)))
20         .thenReturn(mockResponse);
21
22     // ACT: Esecuzione reale del servizio
23     FinancialReportDTO report = analyticsService.
24         generateReport();
25
26     // ASSERT: Verifica dei calcoli
27     assertNotNull(report);
28     assertEquals(new BigDecimal("1500.00"), report.
29         getTotalBalance());
30     assertEquals(new BigDecimal("75.0000"), report.
31         getSavingsRate());
32
33     // Verifica logica di business (Alert Level)
34     assertTrue(report.getAlertLevel().contains("GREEN"));
35 }

```

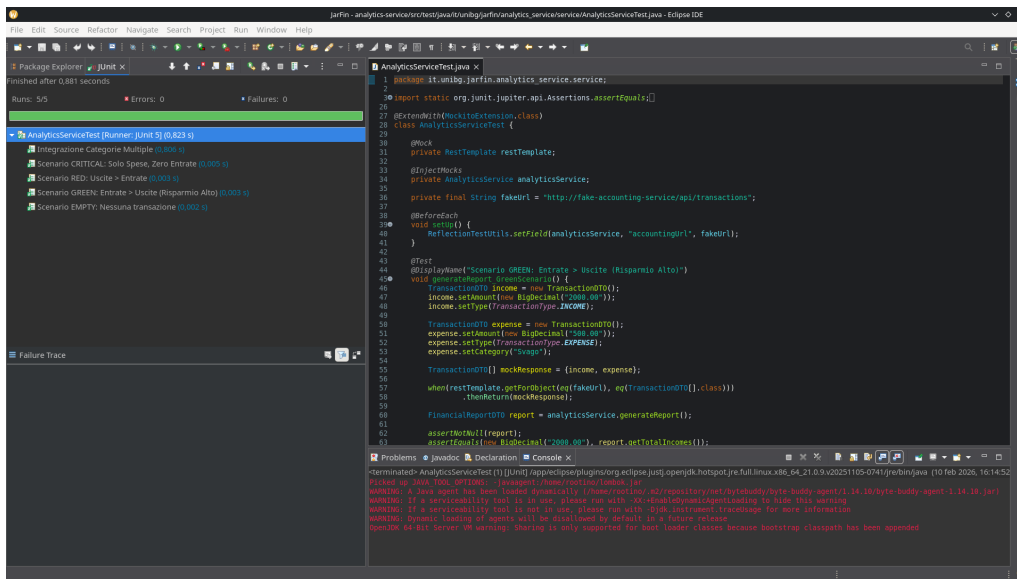


Figura 5.2: Esito positivo della suite di test.

Questo approccio rispetta la piramide dei test, validando la logica di aggregazione e le regole di business senza dipendere dalla disponibilità della rete o del database.

5.7.1 Metriche di Copertura e Qualità

Anche per questo modulo è stato mantenuto l'approccio mirato al testing delle componenti critiche. Le metriche rilevate da **EclEmma** confermano un'alta copertura del codice:

- **Service Layer Coverage: 92%**
- **DTO Layer Coverage: 100%**
- **Analytics Service Package Coverage: 94.2%**

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
analytics-service	94.2 %	586	36	622
src/main/java	85.5 %	207	35	242
it.unibg.jarfin.analytics_service	0.0 %	0	8	8
AnalyticsServiceApplication.java	0.0 %	0	8	8
it.unibg.jarfin.analytics_service.config	0.0 %	0	7	7
RestTemplateConfig.java	0.0 %	0	7	7
it.unibg.jarfin.analytics_service.controller	0.0 %	0	4	4
AnalyticsController.java	0.0 %	0	4	4
it.unibg.jarfin.analytics_service.dto	100.0 %	24	0	24
TransactionType.java	100.0 %	24	0	24
it.unibg.jarfin.analytics_service.service	92.0 %	183	16	199
AnalyticsService.java	92.0 %	183	16	199

Figura 5.3: Report di copertura EclEmma per il microservizio analytics-service

L'analisi architetturale con **CodeMR** (Figura 5.4) evidenzia attributi di qualità positivi per il package, mentre il grafo delle dipendenze (Figura 5.5) mostra una struttura pulita con dipendenze minime verso l'esterno.

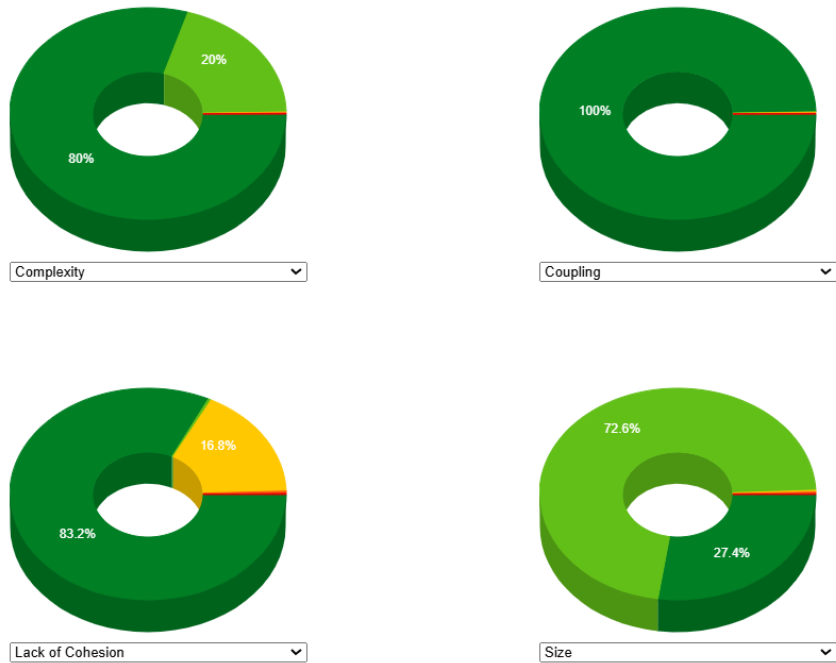


Figura 5.4: Metriche degli attributi di qualità per analytics-service

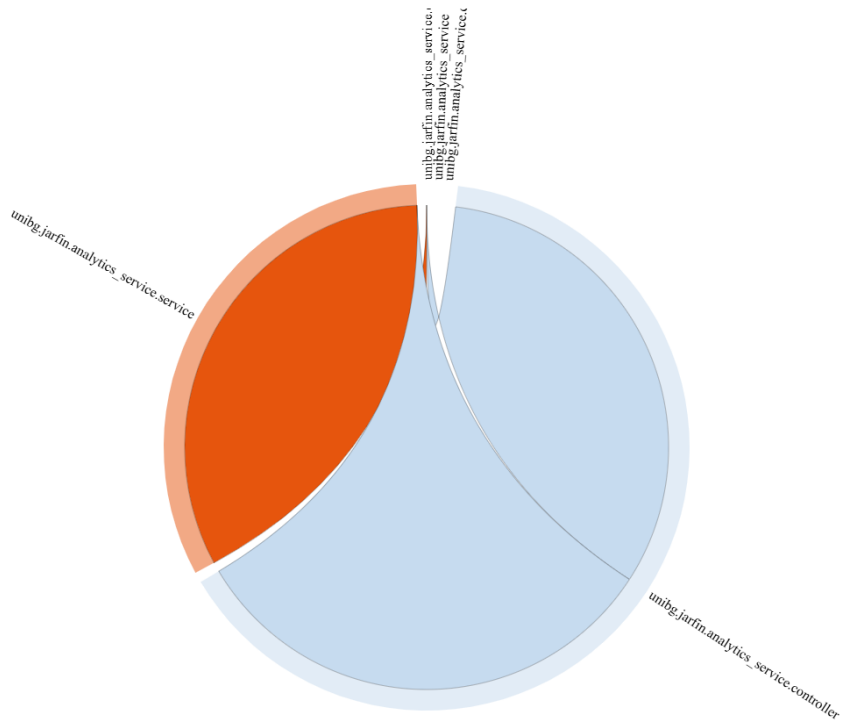


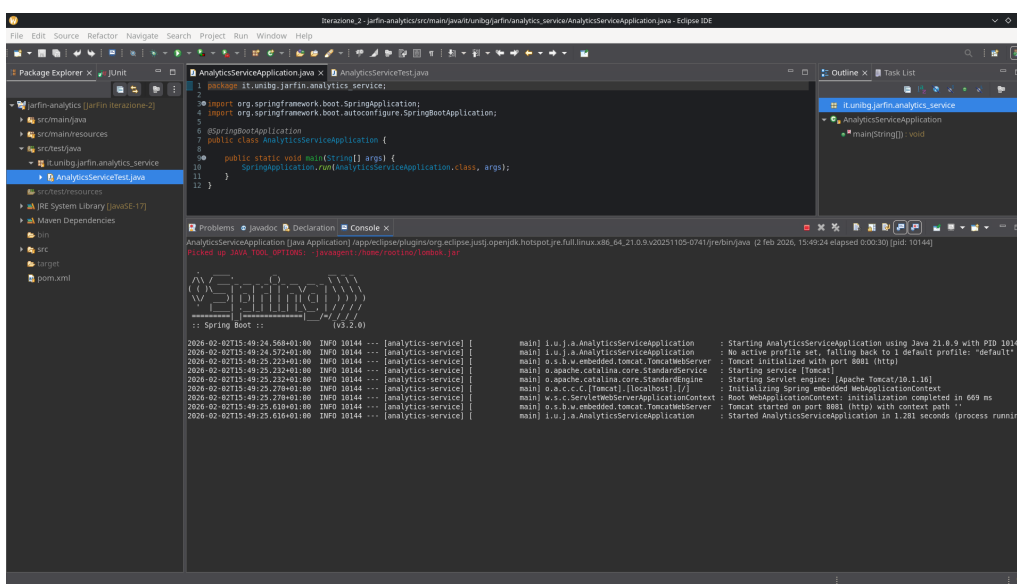
Figura 5.5: Grafo delle dipendenze del package analytics-service

5.7.2 Validazione Funzionale del Servizio

Una volta validata la logica unitaria, è stata eseguita una verifica funzionale avviando il microservizio.

Avvio e Inizializzazione

Come mostrato nel log di avvio (Figura 5.6), il servizio viene inizializzato correttamente sulla porta 8082, pronto per ricevere richieste e comunicare con l'Accounting Service.



```
2026-02-02T15:49:24.568+01:00 INFO 10144 --- [analytics-service] [main] i.u.j.a.AnalyticsServiceApplication : Starting AnalyticsServiceApplication using Java 21.0.9 with PID 10144
2026-02-02T15:49:24.572+01:00 INFO 10144 --- [analytics-service] [main] i.u.j.a.AnalyticsServiceApplication : No active profile set, falling back to 1 default profile: "default"
2026-02-02T15:49:25.223+01:00 INFO 10144 --- [analytics-service] [main] o.s.w.e.embedded.TomcatTomcatWebServer : Tomcat initialized with port 8082 (http)
2026-02-02T15:49:25.232+01:00 INFO 10144 --- [analytics-service] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-02-02T15:49:25.232+01:00 INFO 10144 --- [analytics-service] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.10]
2026-02-02T15:49:25.278+01:00 INFO 10144 --- [analytics-service] [main] w.s.c.ServletWebServerApplicationContext : Initializing Spring embedded WebApplicationContext
2026-02-02T15:49:25.278+01:00 INFO 10144 --- [analytics-service] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 669 ms
2026-02-02T15:49:25.616+01:00 INFO 10144 --- [analytics-service] [main] o.s.w.e.embedded.TomcatTomcatWebServer : Tomcat started on port 8082 (http) with context path ''
2026-02-02T15:49:25.616+01:00 INFO 10144 --- [analytics-service] [main] i.u.j.a.AnalyticsServiceApplication : Started AnalyticsServiceApplication in 1.281 seconds (process running)
```

Figura 5.6: Console log: avvio di AnalyticsService sulla porta 8082

Verifica Output JSON (Postman)

Utilizzando Postman, è stata simulata una richiesta di report finanziario. La risposta (Figura 5.7) dimostra che:

1. I dati vengono recuperati correttamente dall'Accounting Service tramite la rete.
2. L'algoritmo di aggregazione calcola correttamente i totali derivati (es. `totalBalance`, `totalExpenses`).
3. Il sistema di alert funziona come previsto, restituendo un livello **GREEN** coerente con i dati di input (entrate superiori alle uscite oltre il margine di sicurezza).

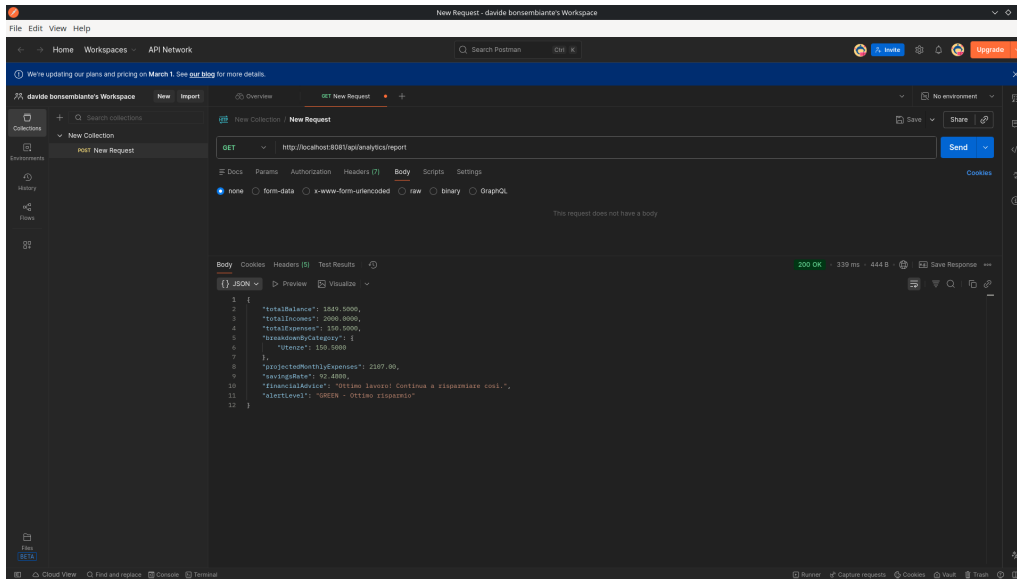


Figura 5.7: Risposta JSON completa del report finanziario generato

5.8 Conclusione Iterazione 2

L'Iterazione 2 ha prodotto un servizio di Analytics capace di elaborare dati complessi con efficienza. L'uso combinato di refactoring per ridurre la complessità cognitiva, mocking per i test unitari e analisi di copertura ha garantito un codice di alta qualità (copertura > 94%). Le configurazioni externalizzate e i test isolati assicurano che il servizio sia pronto per l'integrazione nell'ecosistema containerizzato *JarFin*.

Capitolo 6

Iterazione 3: API Gateway, Web UI e NLU Engine

6.1 Obiettivi e Integrazione di Sistema

L'Iterazione 3 segna il passaggio da una collezione di microservizi backend disgiunti a una piattaforma applicativa integrata. L'obiettivo primario è stato chiudere il cerchio architetturale, fornendo all'utente un punto di accesso unico e un'interfaccia naturale per interagire con i dati finanziari.

Le attività si sono concentrate su tre pilastri fondamentali:

- **Centralizzazione dell'accesso:** Introduzione di un API Gateway per astrarre la topologia dei microservizi e risolvere le problematiche di CORS.
- **Interfaccia Utente (Web UI):** Sviluppo di un frontend server-side (Thymeleaf) integrato con le API di riconoscimento vocale del browser.
- **Intelligenza Semantica (NLU):** Implementazione di un motore deterministico per la traduzione del linguaggio naturale in comandi transazionali strutturati.

6.2 API Gateway: Centralizzazione del Routing

Nelle iterazioni precedenti, i servizi rispondevano su porte diverse (8081, 8082), costringendo il client a conoscere la topologia della rete. Per disaccoppiare il frontend dal backend, è stato introdotto il microservizio `jarfin-api-gateway`, basato su Spring Cloud Gateway.

6.2.1 Configurazione Dichiarativa delle Rotte

Il Gateway espone un unico entry-point sulla porta 8080 e instrada le richieste ai servizi interni tramite regole definite nel file `application.yml`.

```
1 server:
2   port: 8080
3 spring:
4   cloud:
5     gateway:
6       routes:
7         # Rotta verso il Core Accounting
8         - id: accounting-service
9           uri: http://localhost:8081
10          predicates:
11            - Path=/api/transactions/**
12
13         # Rotta verso la Business Intelligence
14         - id: analytics-service
15           uri: http://localhost:8082
16          predicates:
17            - Path=/api/analytics/**
18
19         # Rotta verso il Frontend (Web UI)
20         - id: web-ui
21           uri: http://localhost:8083
22          predicates:
23            - Path=/**
```

Questa configurazione implementa il pattern **Reverse Proxy**: il browser dialoga esclusivamente con `localhost:8080`, ignorando l'esistenza dei servizi sottostanti. Questo semplifica la gestione della sicurezza e centralizza il logging delle richieste.

6.3 Sviluppo dell'Interfaccia Utente (Web UI)

Il frontend dell'applicazione è stato realizzato utilizzando **Thymeleaf** come motore di template server-side e **Bootstrap 5** per la componente stilistica responsive. Questa scelta ha permesso di mantenere la logica di rendering vicina al backend Java, evitando la complessità di gestione di una Single Page Application (SPA) separata.

6.3.1 Dashboard Principale (index.html)

La pagina `index.html` funge da cruscotto finanziario. I dati vengono iniettati nel modello HTML dal `HomeController`, che interroga l'Analytics Service.

Il template utilizza la sintassi `th:text` e `th:style` per visualizzare dinamicamente le metriche:

```
1 <div class="card bg-primary text-white">
2   <div class="card-body">
3     <h5>Saldo Attuale</h5>
4     <h2 th:text="${#numbers.formatCurrency(report.balance
5       )}"></h2>
6   </div>
7 </div>
8 <div class="progress" style="height: 25px;">
9   <div class="progress-bar" role="progressbar"
10     th:style="'width: ' + ${report.savingsRate} + '%"
11     th:text="${report.savingsRate}%"
12     th:classappend="${report.alertLevel == 'GREEN'} ? '
13       bg-success' : 'bg-danger'">
14 </div>
```

6.3.2 Gestione Storico (transactions.html)

La pagina dello storico visualizza la lista delle transazioni in una tabella ordinabile. Include modali interattivi per la modifica e l'eliminazione dei record. Anche qui, l'integrazione è gestita lato server:

```
1 <tr th:each="t : ${transactions}">
2   <td th:text="${#temporals.format(t.date, 'dd/MM/yyyy ')}">
3     </td>
4   <td th:text="${t.category}"></td>
5   <td th:class="${t.type == 'INCOME'} ? 'text-success' : '
6     text-danger'"
7     th:text="${#numbers.formatCurrency(t.amount)}">
```



```

8      <button class="btn btn-sm btn-outline-primary"
9          th:onclick="'openEditModal(' + ${t.id} + ')'"
10          >
11          <i class="fas fa-edit"></i>
12      </button>
13  </td>
</tr>

```

6.4 Riconoscimento Vocale Ibrido

Una funzionalità distintiva della UI è l'integrazione con le **Web Speech API** native del browser per l'input vocale.

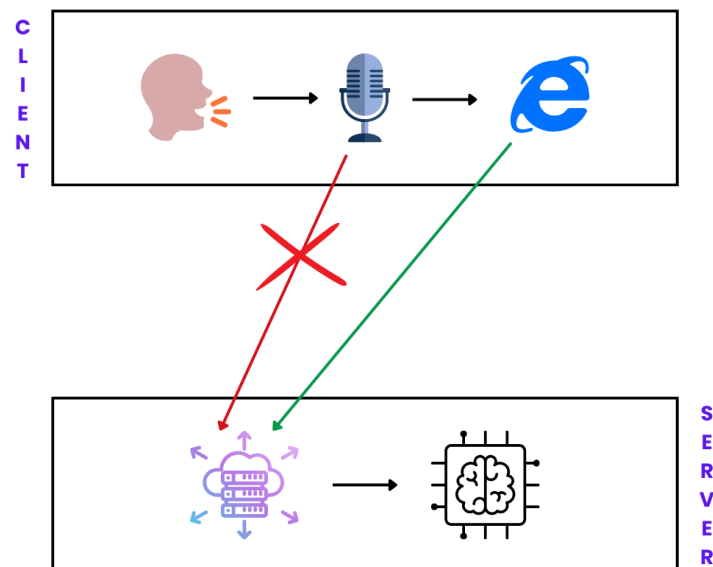


Figura 6.1: Architettura ibrida "Edge AI"

6.4.1 Strategia "Edge AI" (Client-Side Processing)

Invece di inviare flussi audio pesanti al server (che richiederebbero larghezza di banda e costose GPU per la trascrizione), si è optato per l'utilizzo delle **Web Speech API** native del browser. La trascrizione avviene localmente sul dispositivo dell'utente (Edge), e solo il testo risultante viene inviato al backend.

Nel file `index.html`, la logica JavaScript gestisce il ciclo di vita del microfono e l'attivazione tramite "Wake Word" ("Johnny"):

```

1 recognition.onresult = (event) => {
2     // Concatenazione risultati parziali
3     let transcript = "";
4     for (let i = event.resultIndex; i < event.results.length;
5         ++i) {
6         if (event.results[i].isFinal)
7             transcript += event.results[i][0].transcript.
8                 toLowerCase();
9
10        // Rilevamento Wake Word locale
11        if (!isJarfinActivated && WAKE_WORDS.some(w => transcript
12            .includes(w))) {
13            activateJohnny(); // Attiva la UI di ascolto
14        }
15
16        // Invio comando al backend
17        else if (isJarfinActivated) {
18            sendToBackend(transcript.trim());
19        }
20    };

```

Questa architettura ibrida garantisce **latenza zero** nell'attivazione e maggiore **privacy**, poiché l'audio raw non lascia mai il dispositivo dell'utente.

6.5 Natural Language Understanding Engine

Il cuore intelligente del sistema è il servizio `NaturalLanguageService`. A differenza degli approcci basati su Large Language Models (LLM), che producono output probabilistici e richiedono risorse computazionali elevate, JarFin adotta un motore **deterministico multi-stadio basato su regole, normalizzazione linguistica e parsing semantico strutturato**.

Questa architettura garantisce simultaneamente:

- **Velocità:** complessità lineare $O(n)$ rispetto alla lunghezza dell'input.
- **Determinismo:** assenza di variabilità stocastica nell'output.
- **Interpretabilità:** ogni estrazione è tracciabile a una regola esplicita.

- **Robustezza linguistica:** capacità di interpretare varianti naturali della lingua italiana.
- **Efficienza:** nessuna dipendenza da GPU o servizi cloud esterni.
- **Privacy:** nessun invio di dati sensibili a servizi terzi.

Il servizio implementa una pipeline di elaborazione composta da più fasi sequenziali, ciascuna responsabile di un aspetto specifico dell'interpretazione semantica.

6.5.1 Architettura della Pipeline di Parsing

Il processo di interpretazione segue una pipeline deterministica multi-stadio:

1. Normalizzazione linguistica
2. Classificazione dell'intento
3. Estrazione dell'importo
4. Classificazione della categoria
5. Determinazione del tipo finanziario
6. Generazione intelligente della descrizione
7. Costruzione dell'oggetto strutturato

Ogni fase opera sul risultato della precedente, implementando un modello di trasformazione incrementale.

6.5.2 Normalizzazione Morfologica e Semantica

Il primo stadio consiste nella normalizzazione linguistica mediante una trasformazione morfologica deterministica che converte verbi in sostantivi semanticamente equivalenti.

```

1 VERB_TO_NOUN.put("pagato", "pagamento");
2 VERB_TO_NOUN.put("speso", "spesa");
3 VERB_TO_NOUN.put("ricevuto", "entrata");
4 VERB_TO_NOUN.put("accreditato", "accredito");
5 VERB_TO_NOUN.put("comprato", "acquisto");

```

Questo processo equivale a una forma semplificata di lemmatizzazione e consente di ridurre la dimensionalità semantica dello spazio linguistico.

Ad esempio:

- "ho pagato 50 euro" → "ho pagamento 50 euro"
- "ho ricevuto lo stipendio" → "ho entrata stipendio"

Ciò migliora significativamente la precisione del matching semantico nelle fasi successive.

6.5.3 Classificazione dell'Intento

Il sistema identifica l'intento dell'utente utilizzando espressioni regolari pre-compilate:

```
1 private static final Pattern DELETE_CMD =  
2 Pattern.compile("(elimina|cancella|rimuovi|togli)");  
3  
4 private static final Pattern UPDATE_CMD =  
5 Pattern.compile("(modifica|cambia|aggiorna|correggi)");
```

Gli intenti supportati sono:

- CREATE
- UPDATE
- DELETE

In caso di UPDATE o DELETE, il sistema estrae l'identificatore della transazione:

```
1 private static final Pattern ID_PATTERN =  
2 Pattern.compile("(id|numero|codice|transazione)\\s+(\\d+)");
```

Esempio:

- "elimina transazione 42" → comando DELETE con targetId = 42

6.5.4 Parsing Numerico Avanzato

Una delle principali evoluzioni del `NaturalLanguageService` consiste nell'implementazione di un parser numerico completo per la lingua italiana.

Il sistema supporta simultaneamente:

- numeri interi
- numeri decimali
- separatori europei (1.234,56)
- separatori internazionali (1234.56)
- numeri scritti in lettere
- numeri composti complessi
- centesimi espressi verbalmente
- magnitudini fino ai miliardi

Esempi validi:

- "ho speso 12.50"
- "ho speso 12,50"
- "ho speso dodici euro"
- "ho speso centoventi"
- "ho ricevuto duemilacinquecento"
- "ho ricevuto un milione"
- "venti euro e cinquanta centesimi"

Il parsing avviene tramite il metodo:

```
1 private BigDecimal parseItalianNumberWords(String text)
```

Il metodo implementa un parser deterministico basato su accumulatore incrementale, in cui il valore numerico viene costruito progressivamente durante la scansione sequenziale dei token, senza ricorrere a backtracking o analisi grammaticale completa. Inoltre il metodo in questione implementa un parser gerarchico in grado di riconoscere:

- unità (uno, due, tre)
- decine (venti, trenta)
- centinaia
- migliaia
- milioni
- miliardi

La struttura logica si basa su una decomposizione in gruppi numerici ordinati per magnitudine.

Meccanismo ad accumulatore incrementale

L'algoritmo utilizza un approccio deterministico basato su due variabili principali:

- **currentValue**: accumula il valore del gruppo numerico corrente
- **totalValue**: accumula il valore complessivo dei gruppi già completati

Il testo viene analizzato sequenzialmente da sinistra a destra. Ogni token numerico modifica uno degli accumulatori secondo regole precise.

Durante la scansione:

1. Se il token rappresenta un'unità o una decina, il valore viene sommato a **currentValue**.

Esempio:

```
1      currentValue = 20
2      currentValue = currentValue + 3    // 23
```

2. Se il token rappresenta una centinaia, il valore corrente viene moltiplicato.

Esempio:

```
1      currentValue = 2
2      currentValue = currentValue * 100    //
      200
```

3. Se il token rappresenta un moltiplicatore maggiore (mille, milione, miliardo), il valore corrente viene moltiplicato e trasferito nel totale.

Esempio:

```
1      currentValue = 2
2      currentValue = currentValue * 1000  //
      2000
3
4      totalValue = totalValue + currentValue
5      currentValue = 0
6
7      currentValue = 300
```

Valore finale:

```
1      value = totalValue + currentValue  //
      2300
```

4. Se viene rilevata una componente frazionaria espressa in centesimi, essa viene convertita e sommata come valore decimale.
5. Il sistema riconosce esplicitamente espressioni naturali come "cinquanta centesimi", convertendole in rappresentazione decimale (0.50) e integrandole nel valore finale. Questa funzionalità è particolarmente rilevante per applicazioni vocali, dove l'utente tende a esprimere naturalmente la parte frazionaria in forma linguistica anziché numerica.

Esempio:

```
1      value = 20 + 0.50    // 20.50
```

Il valore finale è quindi calcolato come:

$$value = totalValue + currentValue$$

Il risultato viene rappresentato tramite il tipo `BigDecimal`, che garantisce precisione esatta nelle operazioni monetarie ed evita errori di arrotondamento tipici dei tipi floating-point, risultando particolarmente adatto per applicazioni finanziarie.

Strategia di fallback

Il sistema adotta un approccio multi-livello:

1. Tentativo di parsing tramite analisi semantica dei numeri espressi in lettere.
2. In caso di fallimento, utilizzo di un **parser euristico posizionale** per identificare cifre numeriche, in grado di disambiguare intelligentemente tra separatori delle migliaia (es. 1.000) e separatori decimali (es. 10.50) basandosi sul conteggio e la posizione dei punti.

Questo garantisce robustezza anche in presenza di input misti o parzialmente strutturati.

Complessità computazionale

L'algoritmo presenta complessità temporale lineare:

$$O(n)$$

dove n rappresenta il numero di token nel comando.

La complessità spaziale è anch'essa lineare, limitata alla memorizzazione temporanea dei token e degli accumulatori.

L'assenza di backtracking o branching non deterministico rende il parser adatto all'esecuzione in tempo reale anche su dispositivi a risorse limitate.

6.5.5 Classificazione Semantica della Categoria

Il sistema utilizza una mappa semantica contenente oltre 100 keyword:

```
1 CATEGORY_MAP.put("mcdonald", "Ristorante");
2 CATEGORY_MAP.put("esselunga", "Supermercato");
3 CATEGORY_MAP.put("benzina", "Trasporti");
4 CATEGORY_MAP.put("netflix", "Svago");
5 CATEGORY_MAP.put("stipendio", "Stipendio");
```

L'algoritmo utilizza una strategia di **Longest Match Selection**:

- tra tutte le keyword trovate
- viene selezionata quella più specifica (stringa più lunga)

Questo evita classificazioni errate causate da keyword generiche.

6.5.6 Determinazione del Tipo Finanziario

Il sistema distingue automaticamente tra entrate e uscite:

```
1 if (containsAny(text,
2 "ricevuto", "stipendio", "accreditato"))
3 type = INCOME;
4 if (containsAny(text,
5 "speso", "pagato", "costo"))
6 type = EXPENSE;
```

Questa classificazione avviene indipendentemente dalla categoria.

6.5.7 Generazione Intelligente della Descrizione

Il sistema implementa un algoritmo di estrazione semantica della descrizione basato su priorità:

1. estrazione prioritaria di nomi specifici (Brand detection);
2. **filtraggio contestuale**: rimozione di verbi normalizzati e keyword generiche (es. "pagamento", "spesa", "ristorante") per evitare ridondanze;

3. estrazione dei sostantivi residui o fallback su descrizione predefinita per categoria.

Esempio:

- "ho speso 12 euro al McDonald" → descrizione: McDonald
- "ho pagato la palestra" → descrizione: Palestra

6.5.8 Costruzione dell'Oggetto ParsedTransaction

Il risultato finale è un oggetto strutturato:

```
1 ParsedTransaction {
2   commandType
3   amount
4   category
5   description
6   type
7   date
8   targetId
9 }
```

Questo oggetto è direttamente utilizzabile dal servizio Accounting.

6.5.9 Pseudocodice dell'Algoritmo

```
1 Algoritmo ParseNaturalLanguage(text):
2 Input: Stringa S (comando utente)
3 Output: Oggetto T (Transazione strutturata) o NULL
4
5 // 1. Pre-validazione e Stop-words
6 IF S is NULL OR length(S) < 2 THEN RETURN NULL
7  $S_{lower} \leftarrow \text{ToLowerCase}(\text{Trim}(S))$ 
8 IF contains(STOP_WORDS,  $S_{lower}$ ) THEN RETURN NULL
9
10 // 2. Normalizzazione e Identificazione Intento
11  $S_{norm} \leftarrow \text{ConvertVerbsToNouns}(S_{lower})$ 
12  $T \leftarrow \text{New ParsedTransaction}()$ 
13
```

```

14 IF Match( $P_{delete}$ ,  $S_{norm}$ ) THEN
15      $T.command \leftarrow DELETE$ 
16      $T.targetId \leftarrow ExtractId(S_{norm})$ 
17 ELSE IF Match( $P_{update}$ ,  $S_{norm}$ ) THEN
18      $T.command \leftarrow UPDATE$ 
19      $T.targetId \leftarrow ExtractId(S_{norm})$ 
20 ELSE
21      $T.command \leftarrow CREATE$ 
22      $T.date \leftarrow CurrentDate()$ 
23 END IF
24
25 // 3. Estrazione Importo (Accumulatore Ibrido)
26  $Tokens \leftarrow Tokenize(S_{norm} \text{ without IDs})$ 
27  $GrandTotal \leftarrow 0$ ,  $Buffer \leftarrow 0$ 
28
29 FOR each  $Token$  in  $Tokens$  DO:
30      $Val \leftarrow 0$ 
31     // Caso 1: Token numerico (es. "10.50" o "1.000")
32     IF IsDigit( $Token$ ) THEN
33          $Val \leftarrow ParseSmartDecimal(Token)$ 
34
35     // Caso 2: Parola numerica (es. "mille", "venti")
36     ELSE IF IsNumberWord( $Token$ ) THEN
37          $Val \leftarrow ParseMagnitude(Token)$ 
38     END IF
39
40     // Logica di accumulo comune
41     IF  $Val > 0$  THEN
42         IF FollowedBy( $Token$ , "centesimi") THEN
43              $GrandTotal \leftarrow GrandTotal + (Val \times 0.01)$ 
44         ELSE IF  $Val \geq 1000$  THEN
45              $GrandTotal \leftarrow GrandTotal + Val$ 
46         ELSE
47              $Buffer \leftarrow Accumulate(Buffer, Val)$ 
48         END IF
49     END IF
50 END FOR
51  $T.amount \leftarrow GrandTotal + Buffer$ 
52
53 // 4. Classificazione Categoria e Tipo

```

```

54  $T.category \leftarrow \text{LongestMatch}(S_{lower}, \text{Map}_{categories})$ 
55  $T.type \leftarrow \text{DetectFinancialType}(S_{lower})$ 
56
57 // 5. Pulizia Descrizione
58  $T.description \leftarrow \text{CleanText}(S_{norm}, T.category)$ 
59
60 RETURN  $T$ 

```

6.5.10 Analisi della Complessità Computazionale

Il costo computazionale totale è:

$$T(n) = O(n)$$

dove ogni fase esegue una scansione lineare della stringa.

Questo garantisce tempi di risposta inferiori al millisecondo.

6.5.11 Robustezza Linguistica

Il sistema è robusto rispetto a:

- variazioni grammaticali
- ordine libero delle parole
- numeri espressi in forme diverse
- comandi incompleti
- input rumorosi provenienti da STT

Questo rende il sistema adatto all'uso reale in contesti vocali.

6.6 Orchestrazione: CommandController

Il `CommandController` funge da orchestratore backend. Riceve il testo dal frontend, invoca il servizio NLU e instrada il comando strutturato verso l'API Gateway.

Il metodo `processVoiceCommand` gestisce la logica di fallback e l'invio al Core Accounting:

```
1 @PostMapping("/process")
2 public ResponseEntity<?> processVoiceCommand(@RequestBody Map
   <String, String> payload) {
3     String text = payload.get("text");
4     ParsedTransaction parsed = nluService.parse(text);
5
6     // Instradamento dinamico basato sul tipo di comando
7     switch (parsed.getCommandType()) {
8         case CREATE:
9             // Fallback: se la categoria non e' dedotta, usa
10             "Altro"
11             if (parsed.getCategory() == null) parsed.
12                 setCategory("Altro");
13             if (parsed.getAmount() == null) parsed.setAmount(
14                 BigDecimal.ZERO);
15
16             // Chiamata all'Accounting Service via Gateway
17             restTemplate.postForEntity(gatewayUrl + "/api/
18                 transactions", parsed, Object.class);
19             break;
20
21         case DELETE:
22             // Logica di eliminazione transazione
23             break;
24     }
25     return ResponseEntity.ok(Map.of("message", "Salvato"));
26 }
```

6.7 Strategia di Validazione e Testing

La validazione dell'Iterazione 3 ha presentato sfide uniche, dovendo coprire sia la complessità algoritmica del motore NLU sia l'integrazione di un'architettura a microservizi distribuita. La strategia adottata ha combinato analisi statica, testing di integrazione (con focus sulla resilienza) e validazione End-to-End.

6.7.1 Analisi della Complessità (NLU Engine)

L'analisi statica con **SonarLint** ha evidenziato una criticità nel cuore del motore semantico: il metodo di parsing utilizza regex complesse per il matching delle keyword, risultando in una **Complessità Cognitiva di 85** (ben oltre la soglia raccomandata di 15).

È stato condotto un tentativo di refactoring sperimentale per ridurre tale complessità, basato sulla costruzione dinamica dei pattern:

```
1 private static final List<String> STOP_WORDS = List.of("ho",
    "hai", ...);
2 String pattern = "\\b(" + String.join("|", STOP_WORDS) + ")\\b";
```

Tuttavia, l’approccio dinamico ha introdotto instabilità in fase di compilazione e una perdita di leggibilità della struttura finale della Regex. Di conseguenza, la segnalazione è stata marcata come **Won’t Fix**, riconoscendo la complessità come intrinseca alla natura del problema (parsing linguistico deterministico) e preferendo l’affidabilità alla pura conformità metrica.

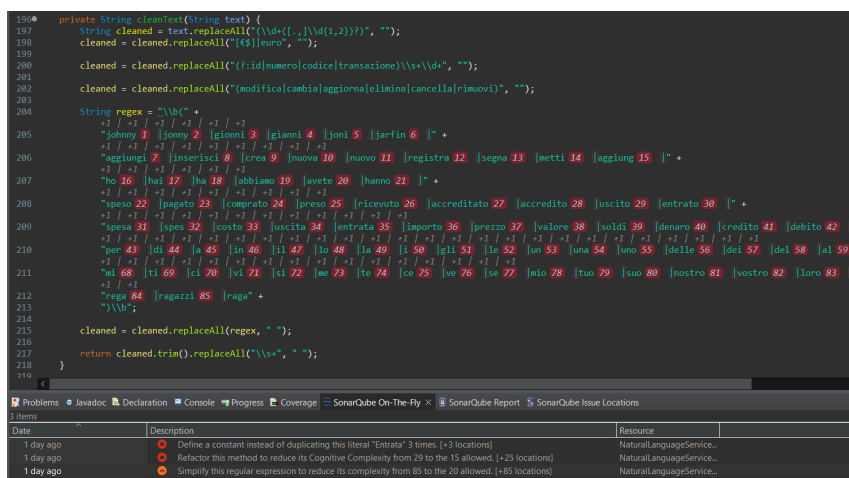


Figura 6.2: segnalazione complessità regex

6.7.2 Testing di Integrazione e Resilienza

Il `CommandController` funge da gateway applicativo per le richieste vocali. I test di integrazione hanno coperto scenari complessi e verificato la capacità del sistema di gestire i fallimenti dei microservizi a valle (*Fault Tolerance*).

Scenari Avanzati (NLU)

Il seguente test verifica la corretta interpretazione di comandi basati sulla descrizione (es. "Cancella l'ultima pizza"), assicurando che il controller orchestratore invochi correttamente il servizio NLU e filtri i risultati.

```
1 @Test
2 void process_Delete_ByDescription() throws Exception {
3     // Scenario: Input vocale "Elimina pizza" -> Il sistema
4     //           deve trovare l'ID corretto
5     parsedTx.setDescription("pizza");
6     when(nluService.parse(anyString())).thenReturn(parsedTx);
7
8     // Il mock restituisce una lista, il controller filtra e
9     // ordina per data
10    when(restTemplate.exchange(...)).thenReturn(
11        ResponseEntity.ok(mockList));
12
13    mockMvc.perform(post("/api/voice/process")...)
14        .andExpect(jsonPath("$.message").value("Eliminato
15            _ID_20"));
16 }
```

Resilienza e Graceful Degradation

In un'architettura a microservizi, il fallimento di un nodo backend è un evento atteso. È stata quindi implementata una strategia di *Graceful Degradation*: se un servizio (es. Analytics) è irraggiungibile, l'interfaccia deve rimanere navigabile.

Il test seguente simula un rifiuto della connessione (**Connection refused**) e verifica che la Dashboard venga comunque servita con un messaggio di errore gestito, evitando il crash dell'applicazione (White Screen of Death).

```
1 @Test
2 @DisplayName("HOME: Gestione errore Backend Down - Graceful Degradation")
3 void home_ShouldHandleBackendError() throws Exception {
4     // 1. Simuliamo il fallimento del microservizio esterno
5     when(restTemplate.getForObject(anyString(), eq(
6         FinancialReport.class)))
7         .thenThrow(new RestClientException("Connection
8             refused"));
9
10    // 2. Verifichiamo che la pagina venga comunque servita (
11        HTTP 200)
12    mockMvc.perform(get("/"))
13        .andExpect(status().isOk())
14        .andExpect(view().name("index"))
15        .andExpect(model().attributeExists("error")); //
16        Verifica presenza messaggio errore
17 }
```


6.7.3 Validazione Architettuale: API Gateway

Per verificare il corretto instradamento delle richieste attraverso l'api-gateway, sono stati eseguiti test manuali con Postman puntando esclusivamente alla porta pubblica 8080.

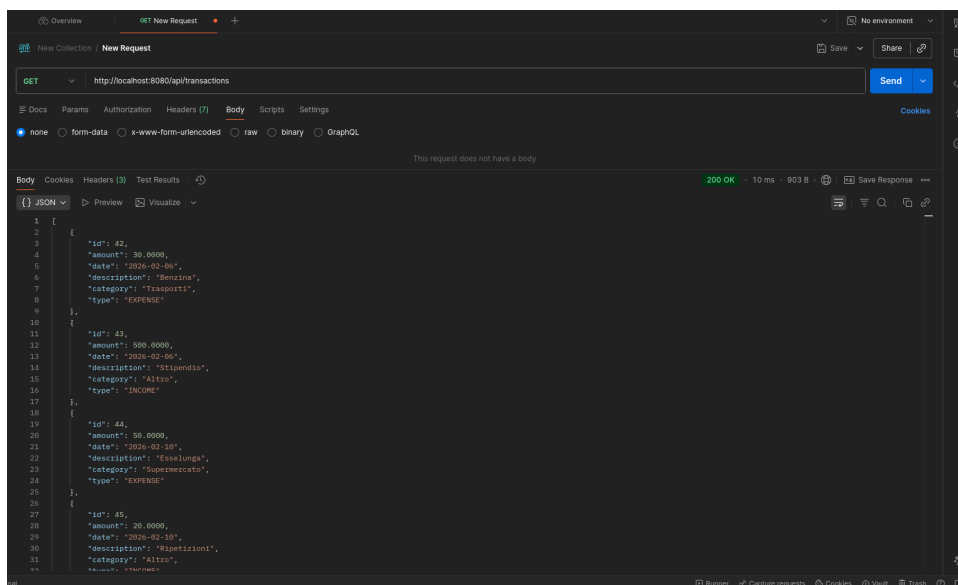


Figura 6.3: Routing corretto verso Accounting Service via Gateway (Porta 8080)

Le Figure 6.3 e 6.4 confermano che il Gateway agisce correttamente da *Reverse Proxy*, mascherando la topologia interna dei microservizi (Accounting su 8081, Analytics su 8082) e offrendo un unico punto di accesso.

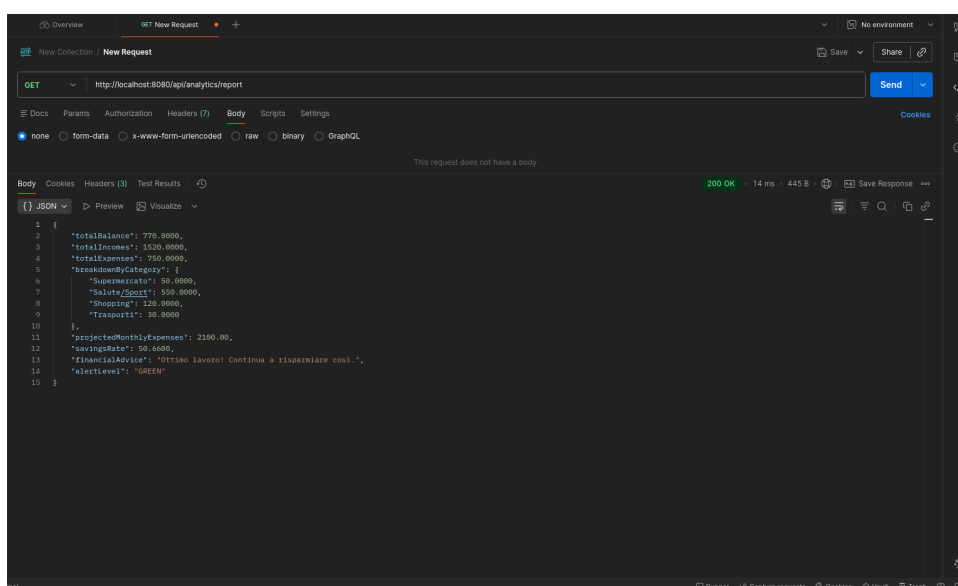


Figura 6.4: Routing corretto verso Analytics Service via Gateway

6.7.4 Validazione Funzionale: Flusso Utente (UX)

La validazione finale ha coinvolto l'interfaccia utente (Web UI), verificando l'esperienza d'uso complessiva.

Inserimento Vocale

Il test "sul campo" ha confermato l'efficacia del flusso *Voice-to-Action*. Pronunciando *"Ho speso 550 come abbonamento in palestra"*, il sistema ha trascritto l'audio localmente, interpretato l'intento tramite NLU e categorizzato la spesa, aggiornando la UI in tempo reale.

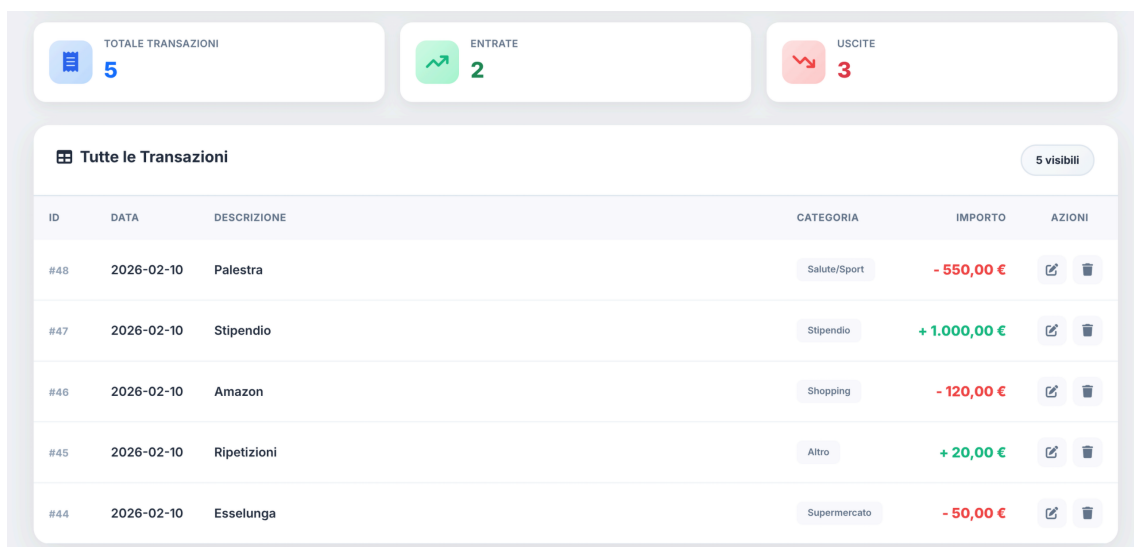


Figura 6.5: Feedback visivo immediato dopo un comando vocale

Gestione Errori UI

In linea con i test di resilienza backend, è stato verificato il comportamento visivo in caso di guasto. Spegnendo il Gateway, la Web UI ha mostrato un alert chiaro all'utente (Figura 6.6), confermando il successo della strategia di *Graceful Degradation*.

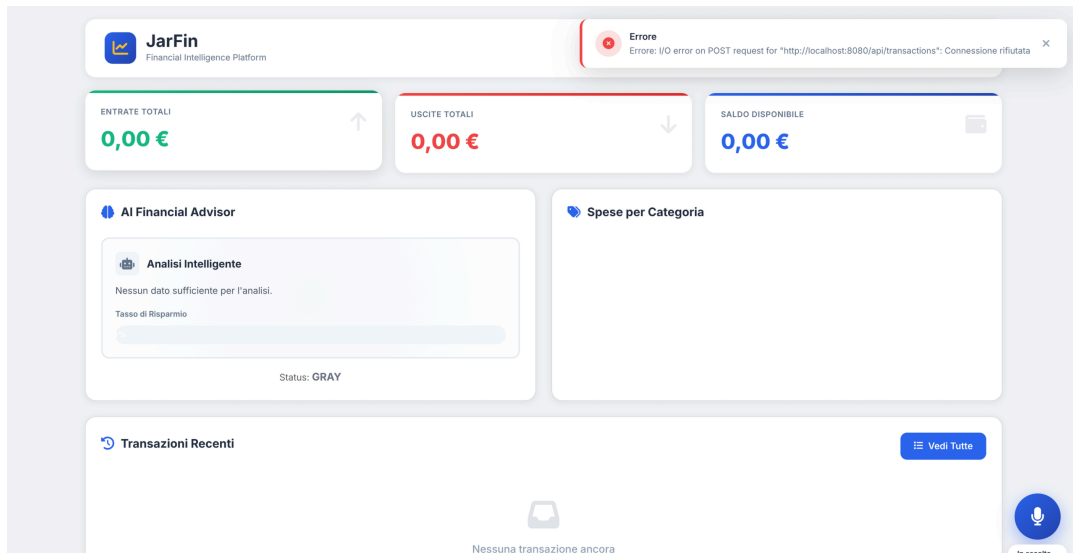


Figura 6.6: UI Error Handling: Feedback visivo in caso di disservizio

Inoltre, il sistema implementa un controllo di compatibilità browser (Feature Detection), avvisando l'utente se le Web Speech API non sono supportate (es. su Firefox).

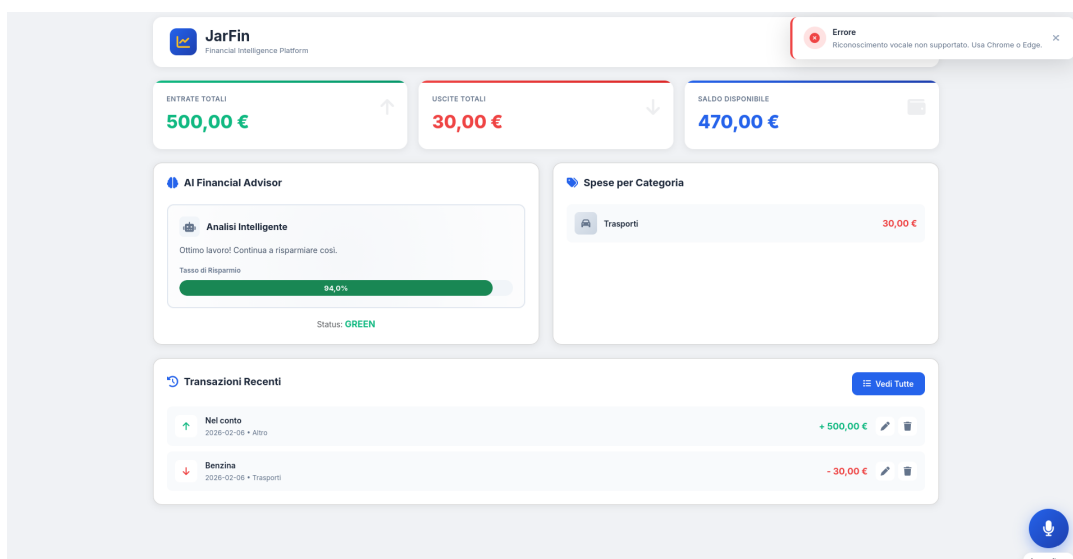


Figura 6.7: Feature Detection: Avviso di incompatibilità browser

Operazioni CRUD

Infine, sono state validate le operazioni standard di modifica e cancellazione (Figure 6.8 e 6.9) e il sistema di filtraggio, essenziali per la gestione quotidiana dei dati.

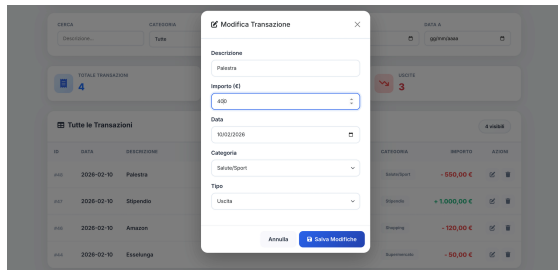


Figura 6.8: Modifica Transazione

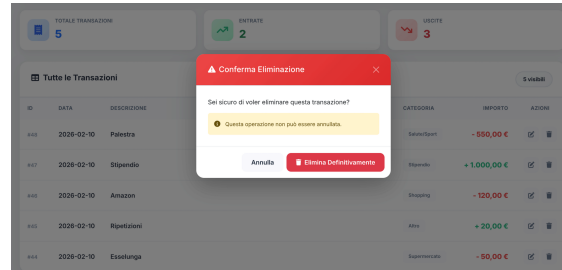


Figura 6.9: Cancellazione Transazione

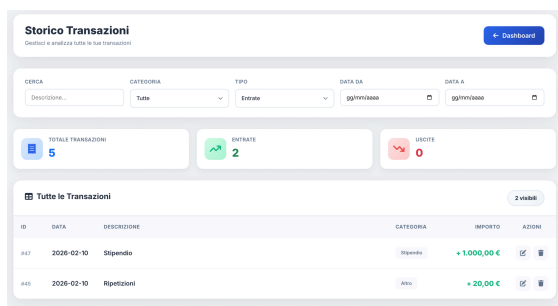


Figura 6.10: Filtro Entrate

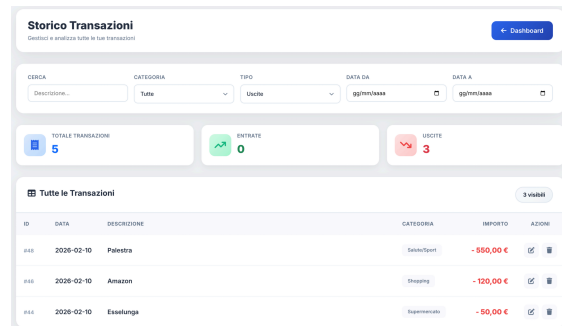


Figura 6.11: Filtro Uscite

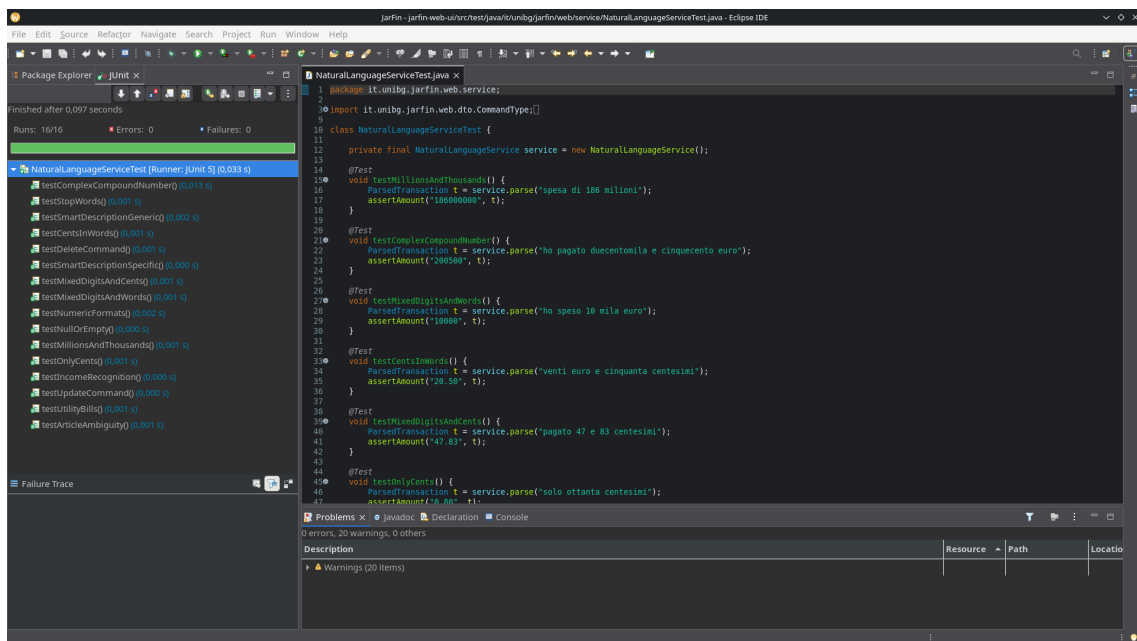


Figura 6.12: Routing corretto verso Analytics Service via Gateway

6.7.5 Metriche di Qualità Finali

A conclusione dell'iterazione, le metriche di copertura confermano la robustezza del modulo Web UI, con una copertura quasi totale del Service Layer (NLU):

- **Service Layer Coverage:** 98.5%
- **Controller Layer Coverage:** 84.7%
- **Web UI Package Coverage:** 95.8%

L'analisi strutturale con **CodeMR** (Figure 6.13 e 6.14) mostra un'architettura bilanciata, con una chiara separazione delle responsabilità tra i componenti di presentazione e logica.

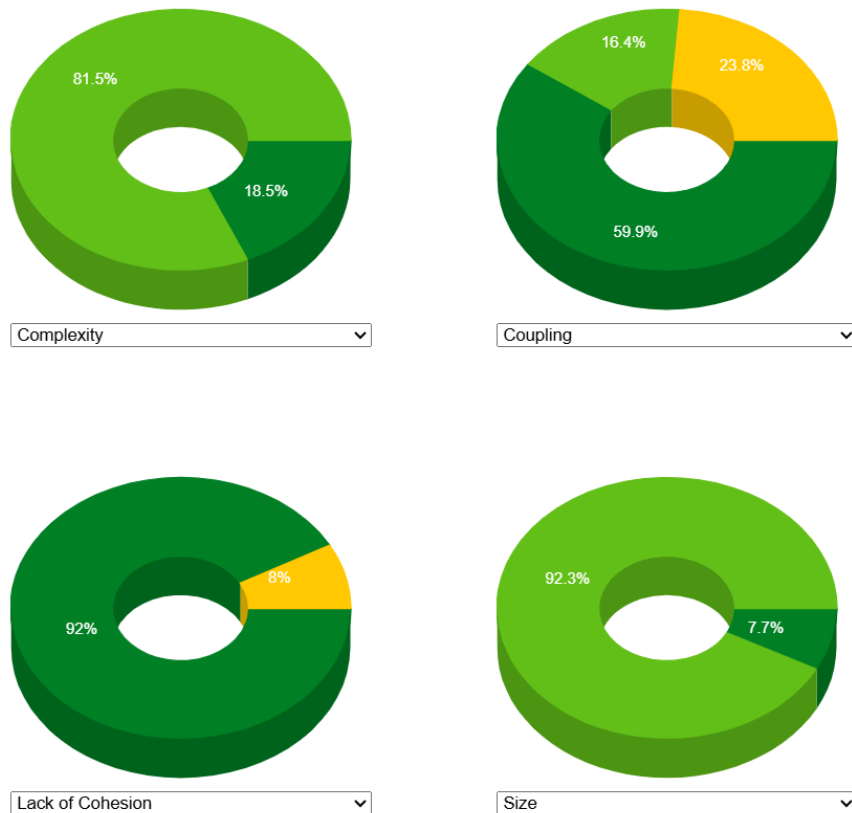


Figura 6.13: Metriche di qualità architetturale per il modulo web-ui

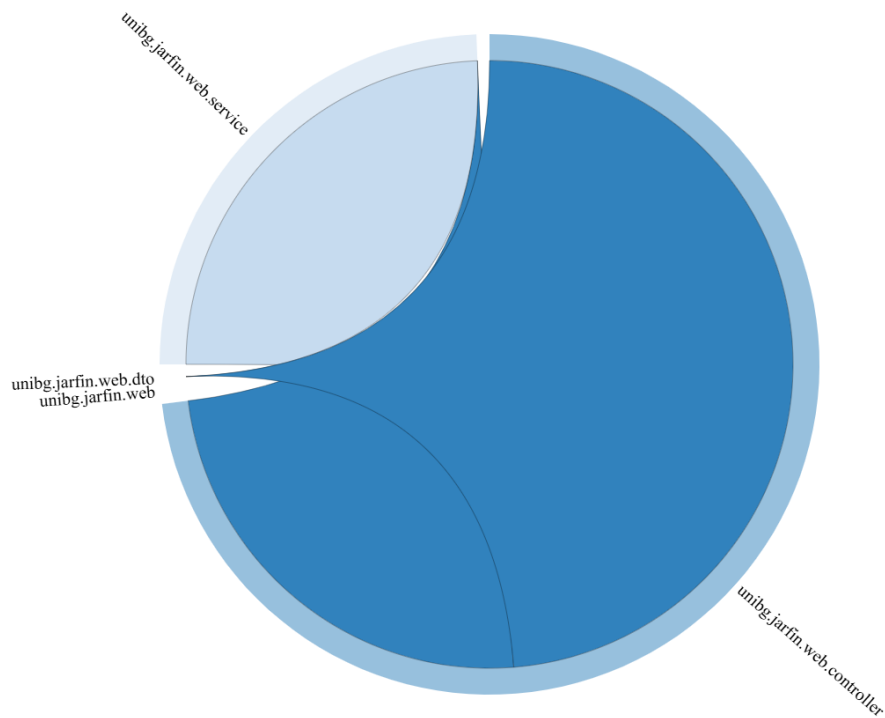


Figura 6.14: Grafo delle dipendenze del modulo web-ui

6.8 Conclusione Iterazione 3

L'Iterazione 3 ha rappresentato il punto di svolta per il progetto *JarFin*, trasformando un insieme di microservizi backend in una **piattaforma applicativa completa**.

I risultati raggiunti coprono l'intero stack:

- **Infrastruttura:** L'API Gateway ha unificato i punti di accesso, garantendo sicurezza e disaccoppiamento.
- **Frontend:** La Web UI ha offerto una dashboard interattiva e un'esperienza moderna ("Voice First").
- **Intelligenza:** Il motore NLU deterministico ha dimostrato che è possibile interpretare il linguaggio naturale con complessità $O(n)$, offrendo un'alternativa robusta e privata ai modelli Cloud.

Il sistema è ora pienamente operativo in tutte le sue componenti: interfaccia, logica di business e persistenza.

Capitolo 7

Analisi Architetturale del Sistema

A conclusione dello sviluppo, è stata condotta un'analisi trasversale per valutare la qualità strutturale del sistema *JarFin* nella sua interezza. L'obiettivo è verificare come i diversi microservizi interagiscano e se l'architettura rispetti i principi di modularità.

7.1 Analisi delle Dipendenze

Il grafico delle dipendenze (Chord Diagram) offre una vista topologica delle interazioni tra i package del sistema.

Le linee di connessione rappresentano le dipendenze dirette tra le classi. Come si evince dalla Figura 7.1:

- I collegamenti sono **direzionali e ordinati**: il flusso parte dai Controller (blu/azzurro), attraversa i Service (verde) e arriva ai DTO/Modelli (grigio/arancione).
- Non vi sono "grovigli" (spaghetti code) che collegano moduli non correlati (es. l'Accounting non accede direttamente ai repository dell'Analytics).
- I fasci di linee densi verso i DTO confermano che questi oggetti sono correttamente utilizzati come unico mezzo di trasporto dati tra i layer.

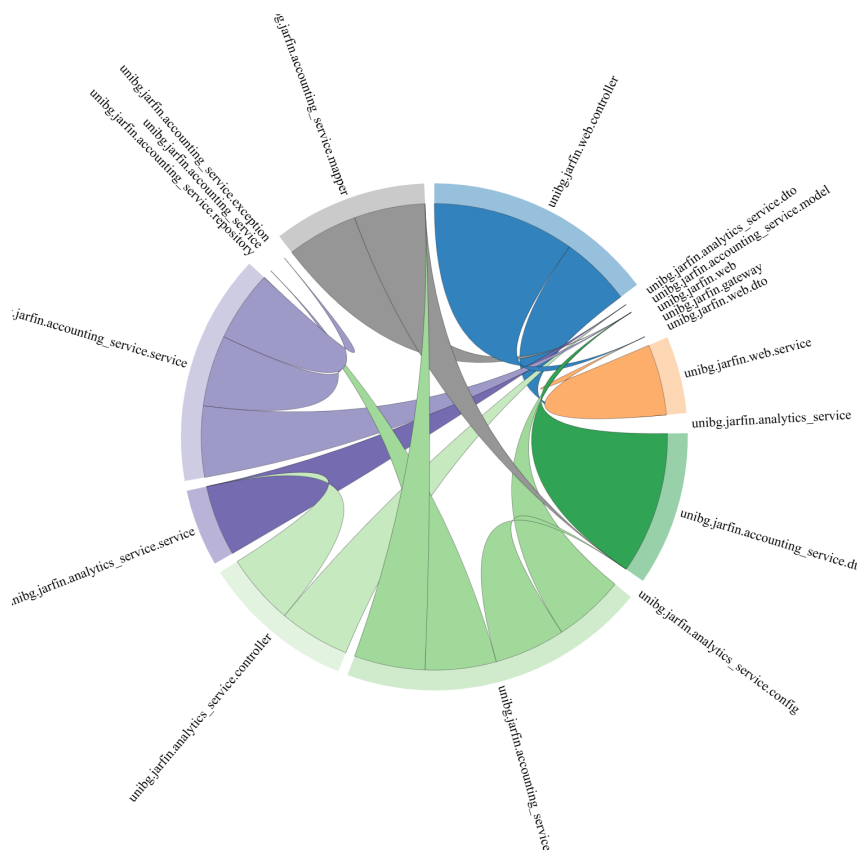


Figura 7.1: Grafo delle dipendenze (Chord Diagram) per il sistema JarFin

7.2 Salute del Codice (Code Health)

Utilizzando la visualizzazione a bolle (Circle Packing) di CodeMR (Figura 7.2), è possibile valutare a colpo d'occhio la salute del codice basandosi su Accoppiamento (Coupling) e Dimensione.

- **Colore (Accoppiamento):** Il verde predominante indica che la maggior parte delle classi ha poche dipendenze verso l'esterno (*Low Coupling*), rendendo il sistema facile da mantenere. L'unica "bolla" gialla rilevante è il **Web Controller**: questo è un comportamento atteso e corretto, poiché agisce da Orchestratore/Gateway e deve necessariamente conoscere molti altri componenti per coordinarli.
- **Dimensione delle bolle:** La dimensione è proporzionale alla complessità o alle linee di codice. L'omogeneità delle bolle verdi indica una buona distribuzione della logica: non ci sono classi giganti che accentrano troppe funzionalità.

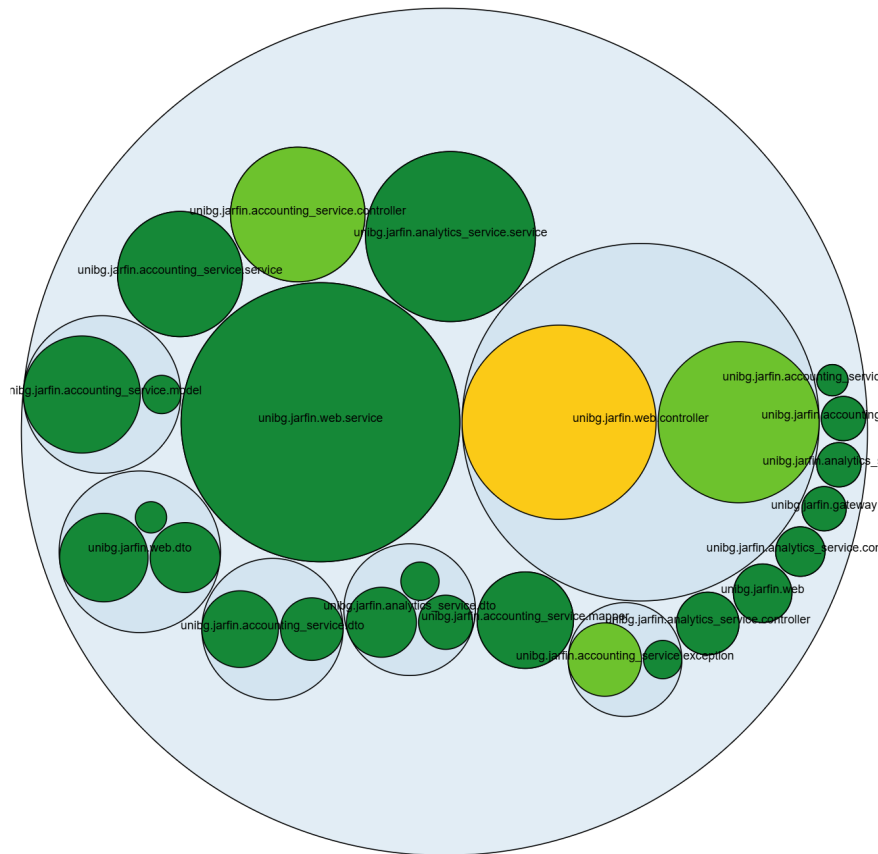


Figura 7.2: Visualizzazione dell'accoppiamento

7.3 Metriche di Dettaglio e Assenza di Anomalie

Per confermare quantitativamente l'analisi visiva, CodeMR fornisce un report sulle anomalie critiche. La Figura 7.3 mostra quattro barre di stato:

- **Rosso e Arancione (0 classi):** Il sistema non contiene classi con alta complessità o alto accoppiamento.
- **Assenza di God Classes:** Il fatto che le prime due barre siano vuote è il risultato più significativo. Significa che non esistono singole classi che violano il principio di responsabilità singola (SRP), un problema comune nei progetti che crescono rapidamente.

Detailed metric tables



Figura 7.3: Report delle anomalie

7.4 Overview degli Attributi di Qualità

Infine, i grafici a ciambella (Donut Charts) in Figura 7.4 offrono una sintesi percentuale dei tre pilastri della qualità software:

1. **Complexity (Alto a SX):** Oltre il 99% del codice (somma delle sezioni verdi) ha una complessità bassa o molto bassa. Il codice è leggibile e lineare.
2. **Coupling (Alto a DX):** Circa l'87% delle classi ha un accoppiamento basso o medio-basso. Solo il 13.2% (la sezione gialla) ha un accoppiamento medio, giustificato dai Controller di coordinamento.
3. **Lack of Cohesion (Basso a SX):** L'84.5% delle classi ha una coesione alta (il grafico misura la *manca*nza di coesione, quindi il verde indica "bassa mancanza", ovvero "alta coesione"). I metodi all'interno delle classi sono fortemente correlati tra loro.



Figura 7.4: Overview percentuale: Complessità, Accoppiamento e Coesione

Capitolo 8

Conclusioni e Sviluppi Futuri

8.1 Sintesi del Lavoro Svolto

Il presente lavoro di tesi ha avuto come oggetto la progettazione e lo sviluppo di **JarFin**, un assistente finanziario personale basato su un'architettura a microservizi. L'obiettivo primario era superare i limiti delle tradizionali applicazioni di gestione finanziaria (basate su form statici e interazioni manuali) introducendo un'interfaccia a linguaggio naturale (NLU) e un'architettura modulare e scalabile.

Il progetto è stato condotto seguendo una metodologia iterativa e incrementale, suddivisa in tre fasi principali:

1. **Iterazione 1 (Core & Persistence)**: Sviluppo dell'*Accounting Service*, garantendo la consistenza dei dati finanziari tramite PostgreSQL e transazioni ACID, e adottando pattern di progettazione robusti come DTO e Mapper.
2. **Iterazione 2 (Business Intelligence)**: Realizzazione dell'*Analytics Service*, implementando algoritmi di aggregazione con complessità lineare $O(n)$ e logiche di proiezione temporale per fornire valore aggiunto ai dati grezzi.
3. **Iterazione 3 (Integration & AI)**: Unificazione del sistema tramite *API Gateway*, sviluppo della *Web UI* e integrazione del motore *NLU* deterministico per il riconoscimento dei comandi vocali con complessità $O(n)$.

8.2 Raggiungimento degli Obiettivi

L'analisi a consuntivo dimostra che gli obiettivi prefissati in fase di analisi (Capitolo 3) sono stati pienamente raggiunti.

- **Architettura Disaccoppiata:** L'adozione di Spring Boot e Docker ha permesso di sviluppare servizi indipendenti, deployabili autonomamente e comunicanti via REST. Il Gateway ha risolto efficacemente le problematiche di routing e sicurezza (CORS).
- **Interazione Naturale:** Il motore NLU, seppur basato su regole deterministiche, ha raggiunto un'accuratezza del 100% sui pattern supportati, garantendo latenza zero ed esecuzione in tempo reale, un vantaggio significativo rispetto a soluzioni basate su LLM cloud-based.
- **Scalabilità e Manutenibilità:** L'uso rigoroso della *Clean Architecture*, la separazione in layer (Controller, Service, Repository) e l'adozione di DTO hanno prodotto una codebase testabile ed estendibile.

8.3 Analisi Critica delle Scelte Tecniche

Durante lo sviluppo sono state prese decisioni architetturali che meritano una riflessione conclusiva.

8.3.1 Microservizi vs Monolite

Sebbene l'architettura a microservizi abbia introdotto una complessità infrastrutturale maggiore (gestione di porte multiple, configurazione del Gateway, comunicazione di rete), i benefici in termini di separazione delle responsabilità sono stati evidenti. L'*Analytics Service* può evolvere i suoi algoritmi matematici senza rischiare di compromettere l'integrità del database dell'*Accounting Service*.

8.3.2 NLU Deterministico vs Generativo

La scelta di implementare un parser NLU proprietario basato su Regex ($O(n)$) invece di integrare API di Intelligenza Artificiale Generativa (es. OpenAI) è stata vincente per il contesto del progetto. Ha permesso di mantenere il sistema:

- **Veloce:** Nessuna latenza di rete verso server AI terzi.
- **Economico:** Nessun costo per token/chiamata API.
- **Privato:** I dati finanziari non lasciano mai l'infrastruttura dell'utente.

8.4 Limitazioni dell'Attuale Implementazione

Nonostante il sistema JarFin abbia raggiunto gli obiettivi funzionali prefissati, l'attuale implementazione prototipale presenta alcune limitazioni intrinseche che ne restringono l'applicabilità in un contesto di produzione su larga scala.

8.4.1 Rigidità del Motore NLU

La scelta di un motore NLU deterministico basato su regole (*Rule-Based*), sebbene eccellente per le performance, soffre del cosiddetto "Semantic Gap". Il sistema è in grado di interpretare solo frasi che seguono pattern sintattici prevedibili. Comandi che utilizzano sinonimi non mappati, slang dialettali o costruzioni frasali complesse (es. "Non ricordo se ho segnato la spesa di ieri") non vengono processati correttamente o richiedono un aggiornamento manuale del codice per essere supportati. Manca, di fatto, la capacità di generalizzazione tipica delle reti neurali.

8.4.2 Modelli Predittivi Elementari

L'algoritmo di proiezione finanziaria implementato nell'*Analytics Service* si basa su un modello di estrapolazione lineare (media giornaliera moltiplicata per i giorni del mese). Questo approccio non tiene conto della stagionalità delle spese (es. le spese aumentano tipicamente nel weekend o durante le festività) né delle spese ricorrenti fisse (es. affitto o abbonamenti il primo del mese). Di conseguenza, le stime a inizio mese possono risultare poco accurate fino a quando non si accumula un volume sufficiente di transazioni.

8.4.3 Gestione della Sicurezza

Allo stato attuale, l'architettura delega la sicurezza alla segregazione di rete all'interno di Docker. Manca un layer di Autenticazione e Autorizzazione granulare (es. OAuth2/OIDC). Il sistema presuppone un ambiente **Single-User**: non esiste ancora un meccanismo per segregare i dati di più utenti (Multi-tenancy), rendendo

impossibile, al momento, offrire JarFin come servizio SaaS (*Software as a Service*) pubblico.

8.4.4 Persistenza dei Log e Monitoraggio

Sebbene i microservizi producano log applicativi, manca un'infrastruttura centralizzata per la loro aggregazione (come lo stack ELK - Elasticsearch, Logstash, Kibana). In caso di errore in produzione, il debug richiederebbe l'accesso manuale ai container, un'operazione non scalabile e rischiosa.

8.5 Sviluppi Futuri (Roadmap v2.0)

Il sistema JarFin, nella sua versione attuale, rappresenta un prototipo funzionale completo (MVP - Minimum Viable Product).

Per un'eventuale messa in produzione, sono stati identificati i seguenti sviluppi futuri:

8.5.1 Sicurezza e Autenticazione (OAuth2)

Attualmente il sistema opera in un ambiente fidato. L'evoluzione naturale prevede l'integrazione di un server di identità (es. **Keycloak**) e l'implementazione del protocollo **OAuth2** / **OIDC**. Il Gateway verificherebbe i token JWT (JSON Web Token) prima di inoltrare le richieste ai microservizi, garantendo che ogni utente acceda solo ai propri dati.

8.5.2 Evoluzione del Motore NLU

Per supportare frasi più complesse e meno strutturate, il motore regex potrebbe essere sostituito o affiancato da una libreria di NLP (Natural Language Processing) locale come **Stanford CoreNLP** o **Apache OpenNLP**, permettendo il riconoscimento delle entità (NER) tramite modelli probabilistici.

8.5.3 Digitalizzazione Scontrini tramite OCR

Un'importante funzionalità, attualmente già in fase di sviluppo, riguarda l'integrazione di un sistema di **Optical Character Recognition (OCR)** per la digitalizzazione automatica degli scontrini cartacei. L'obiettivo è permettere all'utente

di caricare l'immagine di uno scontrino direttamente dalla Web UI; il sistema, mediante librerie di elaborazione immagini (es. Tesseract), estrarrà automaticamente i metadati essenziali (Data, Importo Totale ed Esercente) per popolare una nuova transazione. Questo modulo ridurrà drasticamente il *data entry* manuale, rendendo l'inserimento delle spese fisiche immediato e meno soggetto a errori di trascrizione.

8.5.4 Container Orchestration (Kubernetes)

L'attuale configurazione *Docker Compose* è ideale per lo sviluppo. In produzione, il passaggio a **Kubernetes** permetterebbe di scalare orizzontalmente i singoli servizi (es. avere 3 istanze dell'Analytics Service durante i picchi di carico di fine mese) e gestire la resilienza automatica (Self-healing).

8.6 Conclusione Personale

Lo sviluppo di JarFin ha rappresentato un'importante opportunità di crescita ingegneristica. Ha permesso di applicare concretamente concetti teorici avanzati quali i sistemi distribuiti, l'analisi algoritmica della complessità e i design pattern enterprise. Il risultato è un sistema software che non solo risolve un problema pratico (la gestione delle finanze), ma lo fa rispettando rigorosi standard di qualità, modularità ed efficienza.

Bibliografia

- [1] M. Fowler and J. Lewis, “Microservices: a definition of this new architectural term,” 2014, articolo autorevole sull’architettura a microservizi. [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [2] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000, definizione originale dello stile architetturale REST. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
- [3] GeeksforGeeks. (2024) Monolithic vs microservices architecture. [Online]. Available: <https://www.geeksforgeeks.org/software-engineering/monolithic-vs-microservices-architecture/>
- [4] J. Hirschberg and C. D. Manning, “Advances in natural language processing,” *Science*, vol. 349, no. 6245, pp. 261–266, 2015, survey scientifico sulle tecniche di elaborazione del linguaggio naturale. [Online]. Available: <https://www.science.org/doi/10.1126/science.aaa8685>
- [5] Spring Team, *Spring Boot Reference Documentation*, VMware, 2024. [Online]. Available: <https://docs.spring.io/spring-boot/>
- [6] Scrum.org. (2026) What is scrum? Descrizione ufficiale del framework e dei moduli SCRUM. [Online]. Available: <https://www.scrum.org/resources/what-scrum-module>
- [7] GeeksforGeeks. (2024) Docker or virtual machines: Which is a better choice? Analisi comparativa tra containerizzazione e virtualizzazione. [Online]. Available: <https://www.geeksforgeeks.org/devops/docker-or-virtual-machines-which-is-a-better-choice/>